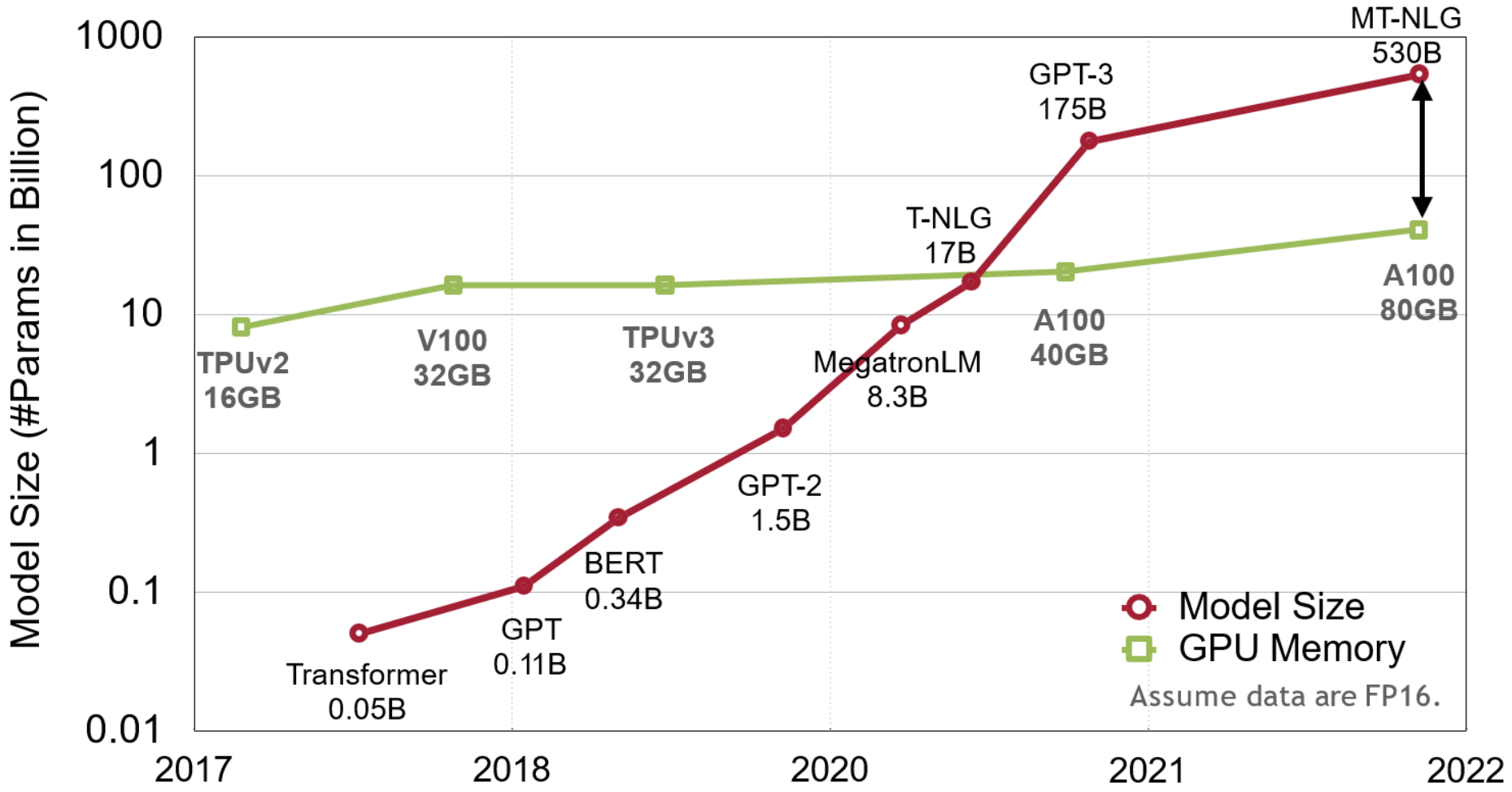
邊緣人工智慧
Edge AI

Quantization

吳凱強
Kai-Chang Wu

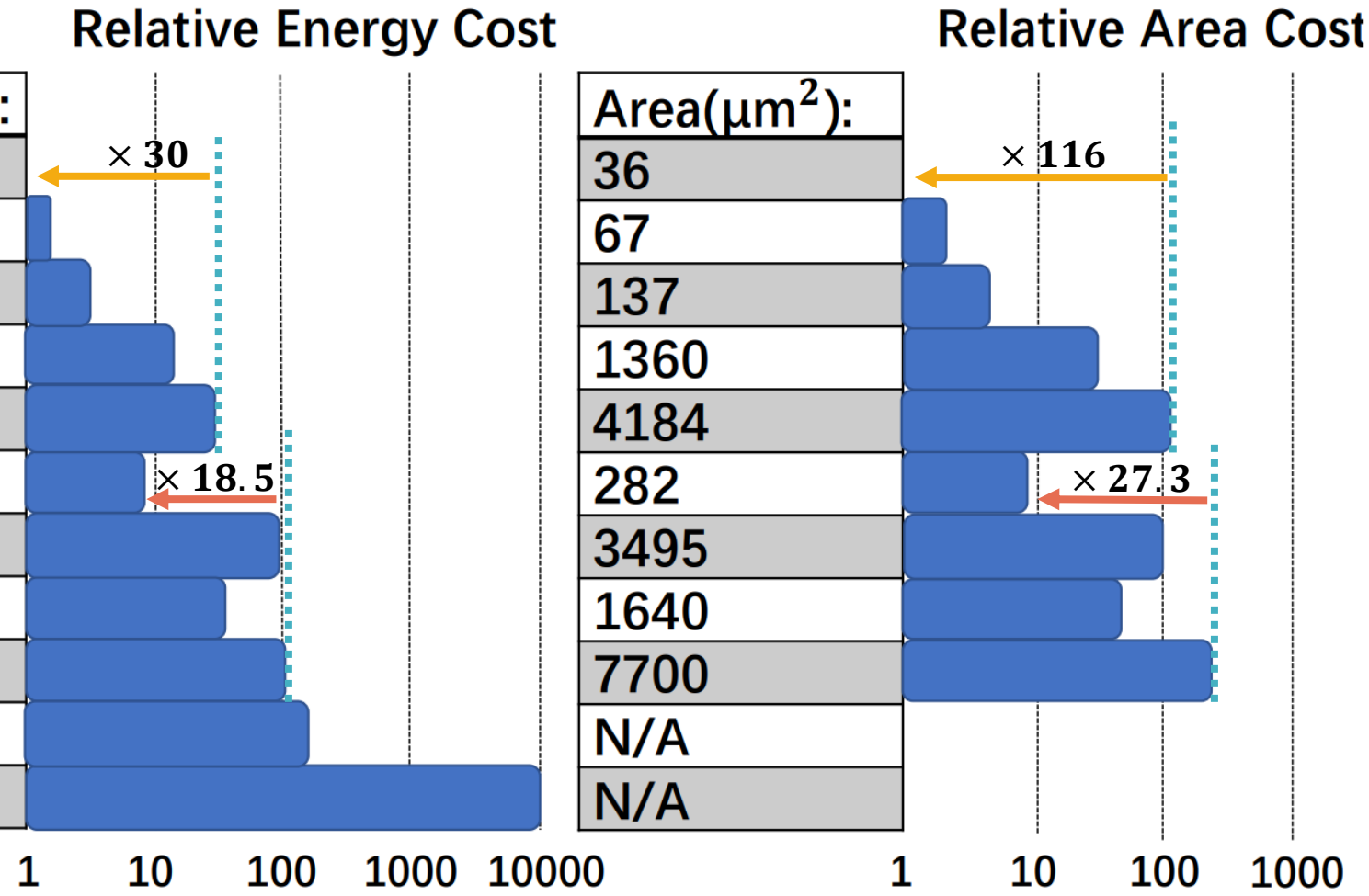


Deep Learning Continues to Scale



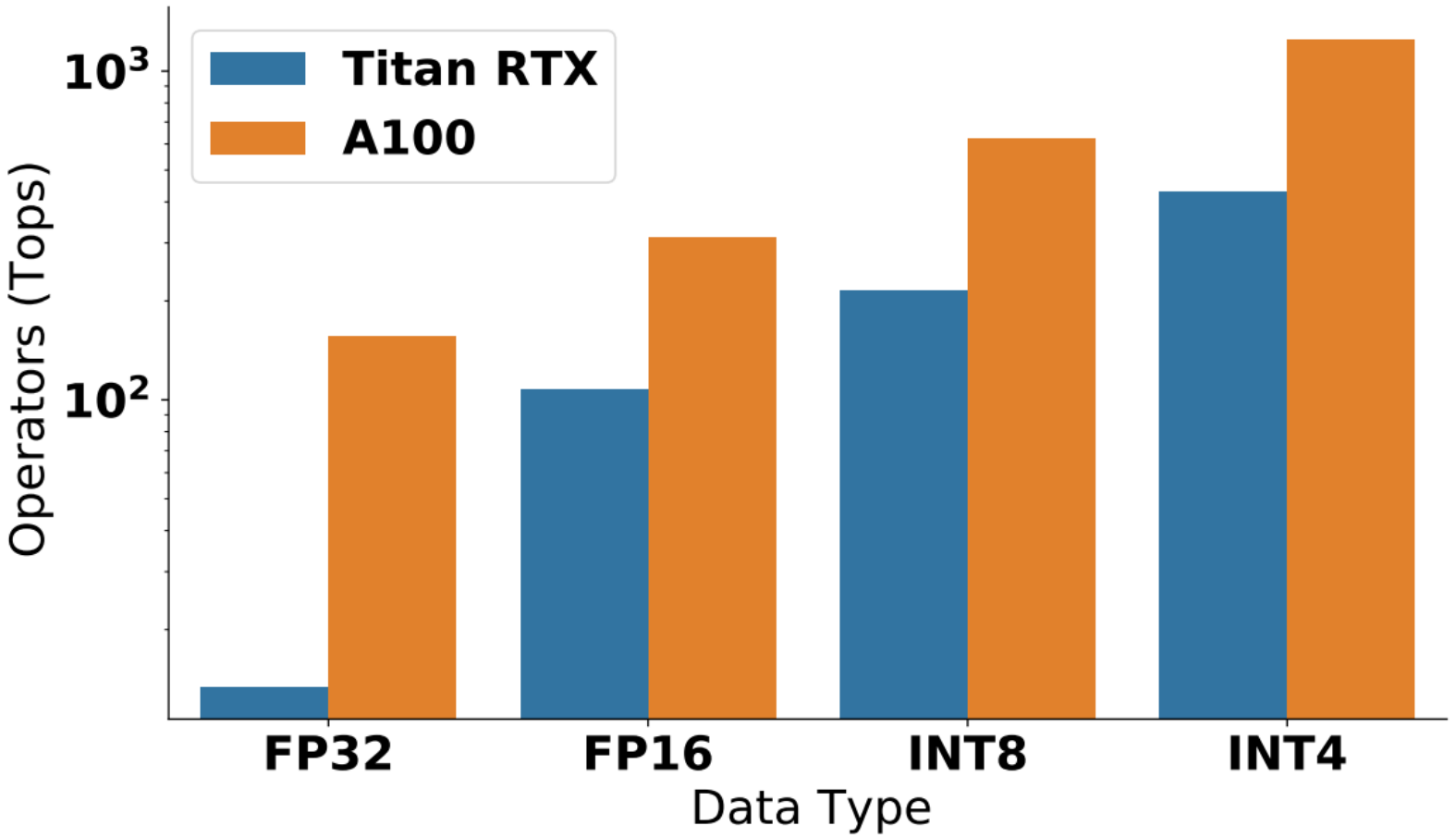
Low Bit-Width Operations are Much Cheaper

Operation:	Energy(pJ):	Area(μm^2):
8b Add	0.03	36
16b Add	0.05	67
32b Add	0.1	137
16b FP Add	0.4	1360
32b FP Add	0.9	4184
8b Mult	0.2	282
32b Mult	3.1	3495
16b FP Mult	1.1	1640
32b FP Mult	3.7	7700
32b SRAM Read (8kb)	5.0	N/A
32b DRAM Read	640	N/A



Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Peak Throughput Comparison: Titan RTX vs. A100



Outline

- (Review) Numeric Data Types
- Introduction to Quantization
- Quantization Methods
  - K-Means-based weight quantization
  - Linear quantization
- Post-Training Quantization (PTQ)
- Quantization-Aware Training (QAT)

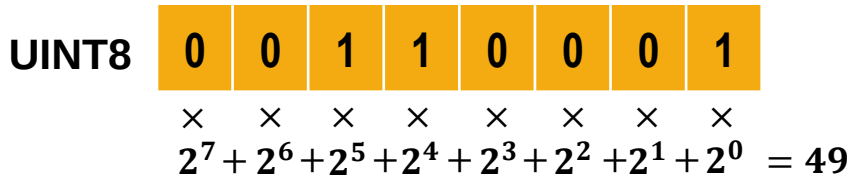


Numeric Data Types

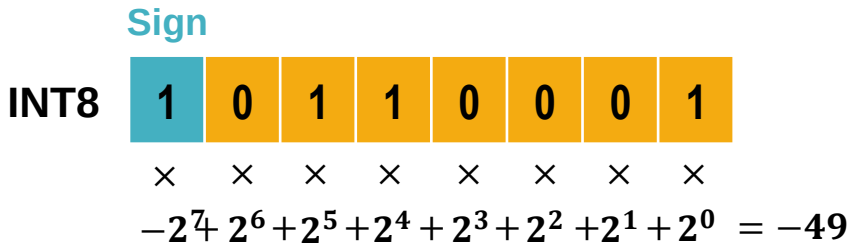


Integer

- Unsigned Integer
  - **n-bit Range:** $[0, 2^n - 1]$



- Signed Integer
  - **n-bit Range:** $[-2^{n-1}, 2^{n-1} - 1]$



Floating-Point Number

- FP32: IEEE 754 standard



Exponent	Fraction = 0	Fraction ≠ 0	
$00_H = 0$	$(-1)^{sign} \times \mathbf{Fraction} \times 2^{\mathbf{1}-127}$		(Exponent = 0)
$01_H \ldots FE_H = 1 \sim 254$	$(-1)^{sign} \times (1 + \mathbf{Fraction}) \times 2^{\mathbf{Exponent}-127}$		(Exponent ≠ 0)
$FF_H = 255$	±INF	NaN	



Floating-Point Number

Exponent width → Range, Fraction Width → Precision

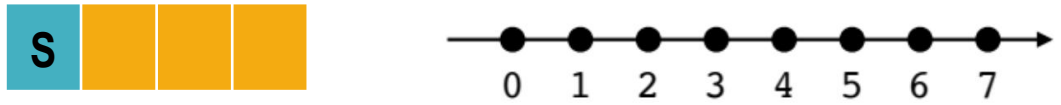
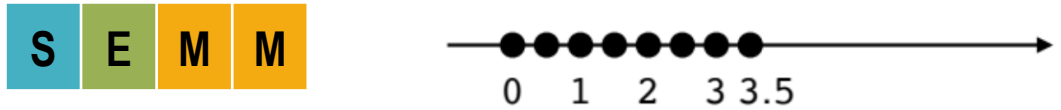
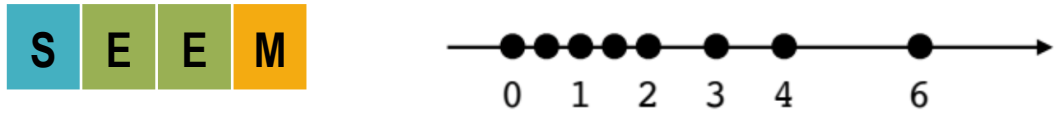
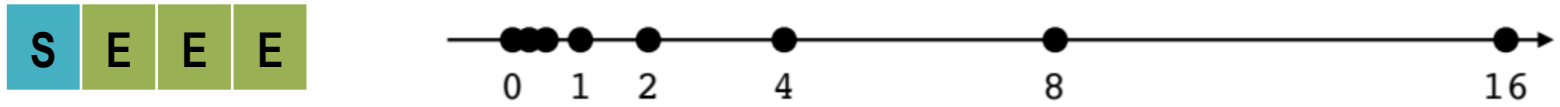
	Total bits	Range
IEEE 754 Single Precision 32-bit Float **(IEEE FP32)**	32	$10^{-38} \sim 10^{38}$
IEEE 754 Half Precision 16-bit Float **(IEEE FP16)**	16	$6 \times 10^{-5} \sim 6 \times 10^{4}$
Google Brain Float **(BF16)**	16	$10^{-38} \sim 10^{38}$







INT4 and FP4

		Total bits	Range
INT4		4	$-8 \sim 7$
FP4 (E1M2)		4	$-3.5 \sim 3.5$
FP4 (E2M1)		4	$-6 \sim 6$
FP4 (E3M0)		4	$-16 \sim 16$

Introduction to Quantization

What is Quantization? (binning)

> **Quantization** is the process of **mapping** a large set of values into a smaller set of **discrete finite values**


Original


2 colors


5 colors


15 colors


100 colors

Original 8-bit



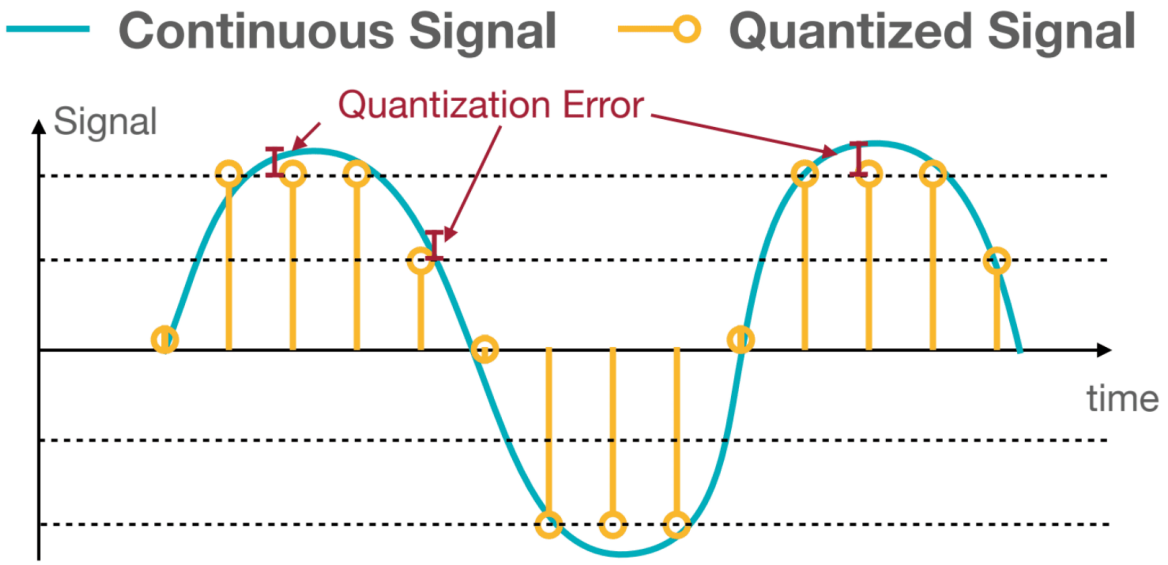
3-bit Quantized



Introduction to Quantization

What is Quantization? (binning)

Quantization is the process of **mapping** a large set of values into a smaller set of **discrete finite values**



The difference between an input value and its quantized value is referred to as quantization error.

Original 8-bit



3-bit Quantized



Quantization Methods

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	**3:**	2.00
1	1	0	3	**2:**	1.50
0	3	1	0	**1:**	0.00
3	1	2	2	**0:**	-1.00

K-Means-based Weight Quantization

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

(− -1) × 1.07

Linear Quantization

Storage	Floating-Point Weights
Computation	Floating-Point Arithmetic

Quantization Methods

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	**3:**	2.00
1	1	0	3	**2:**	1.50
0	3	1	0	**1:**	0.00
3	1	2	2	**0:**	-1.00

K-Means-based Weight Quantization

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

(— -1) × 1.07

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic

K-Means-based Weight Quantization

A look-up table mapping real numbers to integers



K-Means-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49



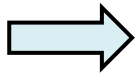
Deep Compression [Han et al., ICLR 2016]

K-Means-based Weight Quantization

weights (32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

cluster



cluster index (2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

indexes

centroids

3:	2.00
2:	1.50
1:	0.00
0:	-1.00

codebook

reconstructed weights (32-bit float)

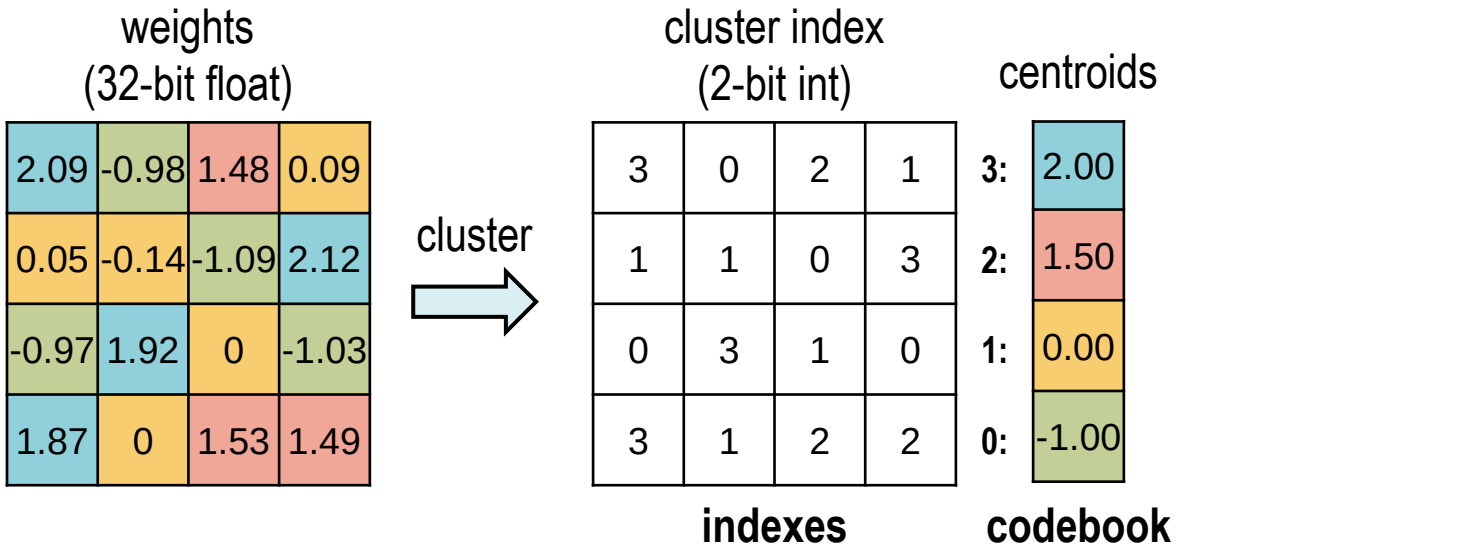
2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
-0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01



K-Means-based Weight Quantization



reconstructed weights
(32-bit float)

2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

Storage: (required bits)

32 bit × 16 = 512 bit

2 bit × 16 = 32 bit + 32 bit × 4 = 128 bit = 160 bit

× 3.2 smaller

Assume N-bit quantization, with #parameters = M >> 2^N :

32 bit × M = 32M bit

× 32/N smaller

N bit × M = NM bit

32 bit × 2^N = 2^{N+5} bit

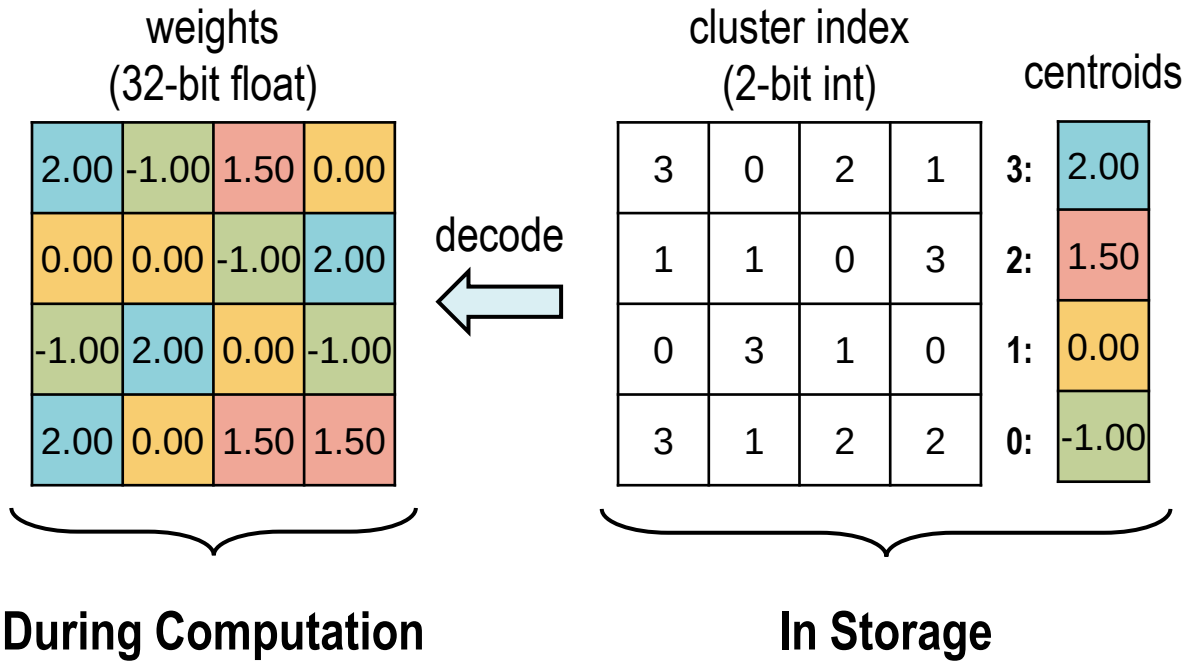
ignore

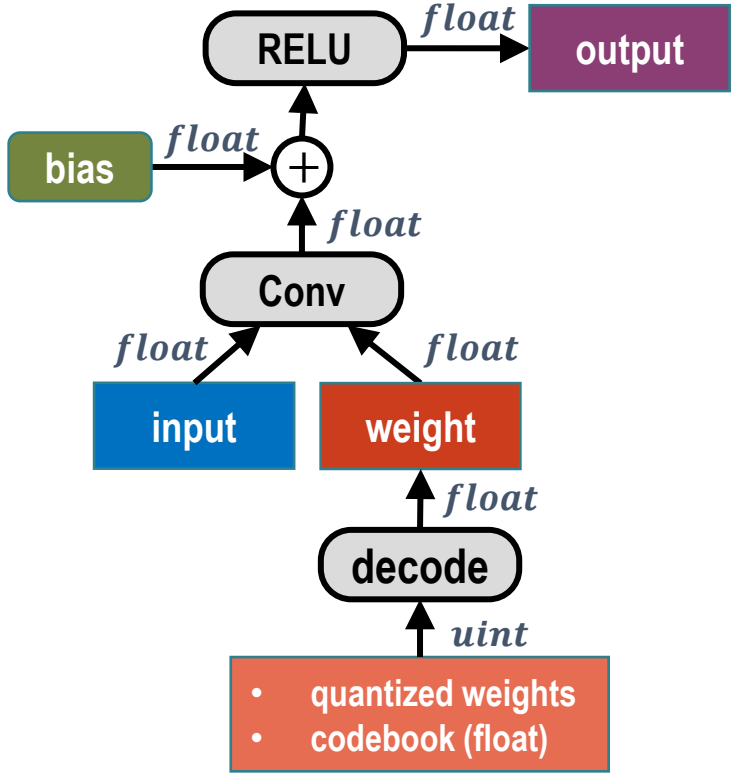
quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
-0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

Deep Compression [Han et al., ICLR 2016]

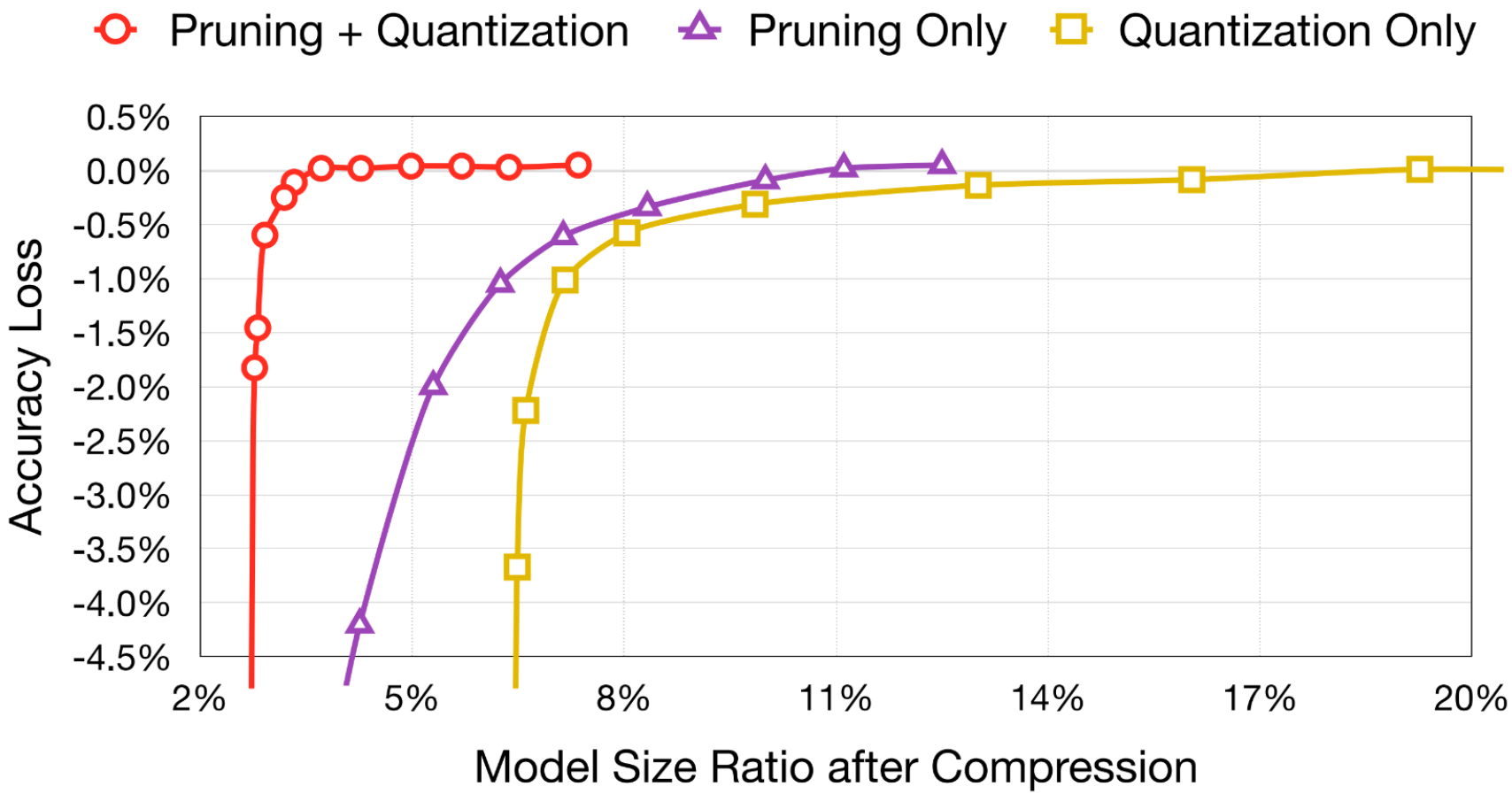
On Inference Time:





- Weights are decompressed using a lookup table (i.e. codebook) on runtime inference.
- "K-Means-based Weight Quantization" only saves storage cost.
  - All computation & memory access are still floating-points.

Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han et al., ICLR 2016]

Quantization Methods

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	**3:**	2.00
1	1	0	3	**2:**	1.50
0	3	1	0	**1:**	0.00
3	1	2	2	**0:**	-1.00

K-Means-based Weight Quantization

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

(− -1) × 1.07

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic

Quantization Methods

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

K-Means-based Weight Quantization

$$\left(\begin{bmatrix} 1 & -2 & 0 & -1 \\ -1 & -1 & -2 & 1 \\ -2 & 1 & -1 & -2 \\ 1 & -1 & 0 & 0 \end{bmatrix} - (-1) \right) \times 1.07$$

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

Linear Quantization

Affine mapping between real numbers and integers



Linear Quantization

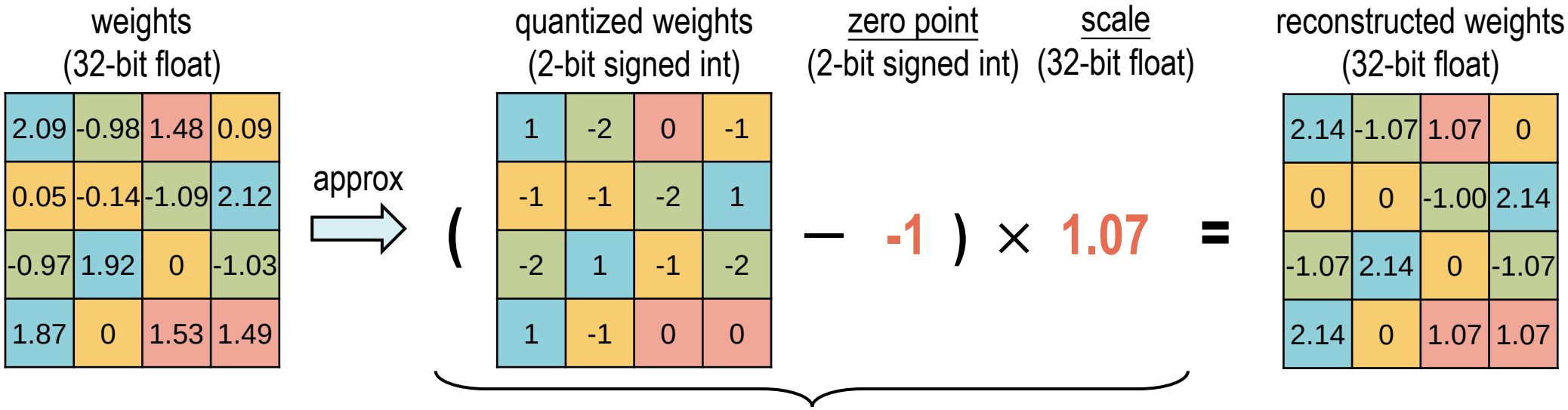
weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

2-bit INT	Decimal
01	1
00	0
11	−1
10	−2



Linear Quantization



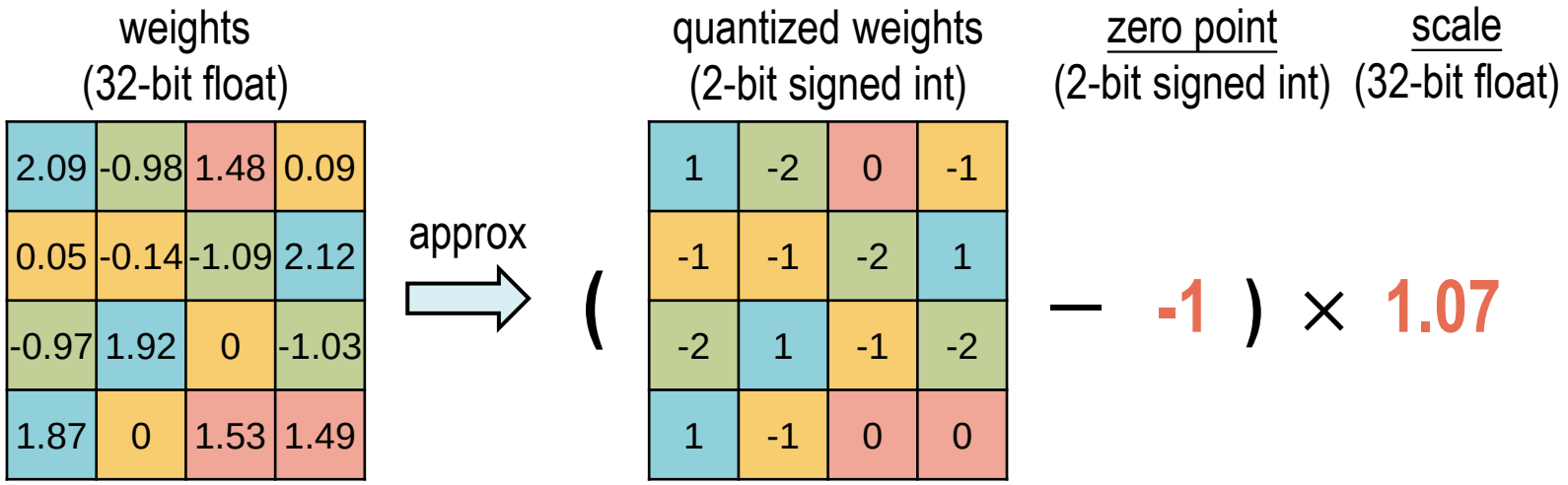
Affine mapping of integers to real numbers

2-bit INT	Decimal
01	1
00	0
11	−1
10	−2

quantization error

-0.05	0.09	0.41	0.09
0.05	-0.14	-0.01	-0.12
0.16	-0.08	0	0.04
-0.27	0	0.46	0.42

Linear Quantization



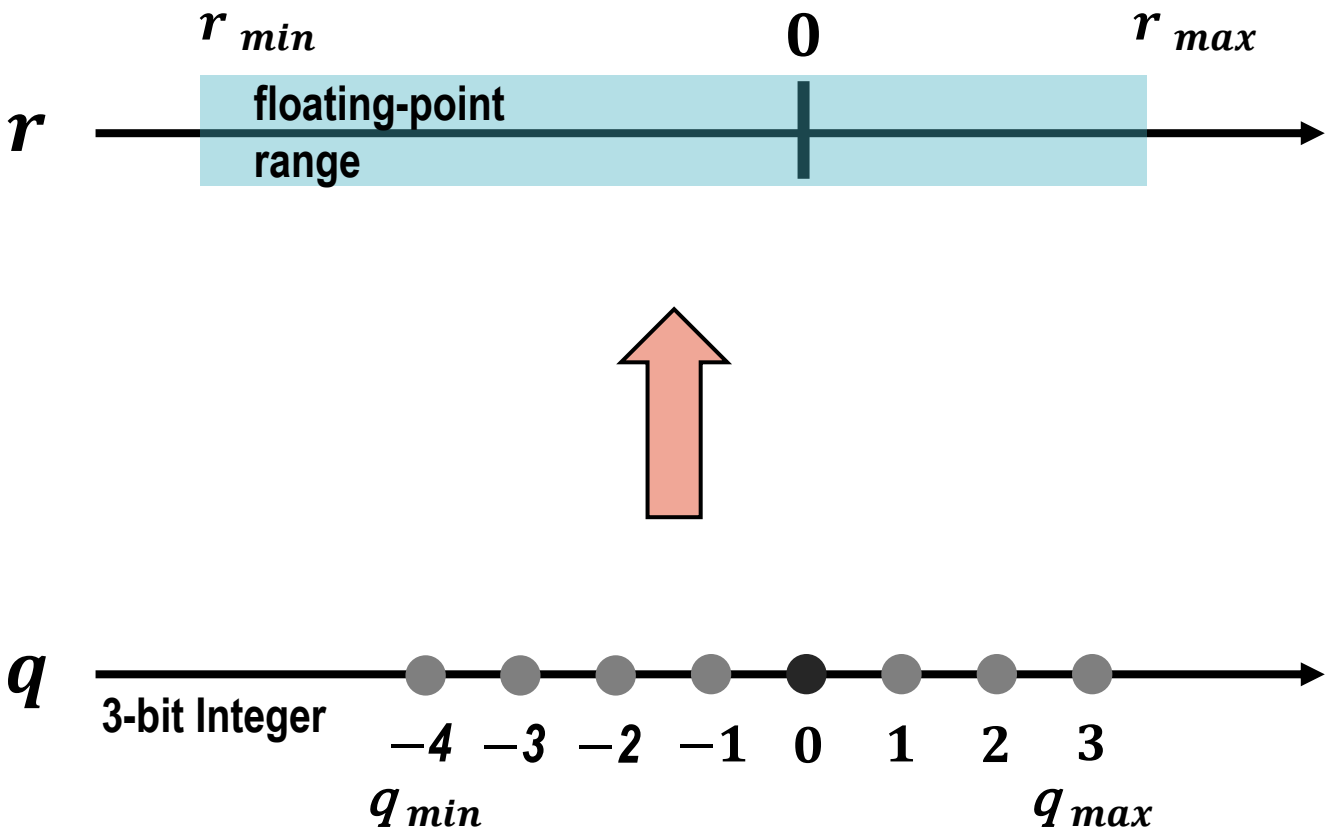
$$r = (q - Z) \times S$$

Floating-point **Integer** **Integer** **Floating-point**

How to determine these values?

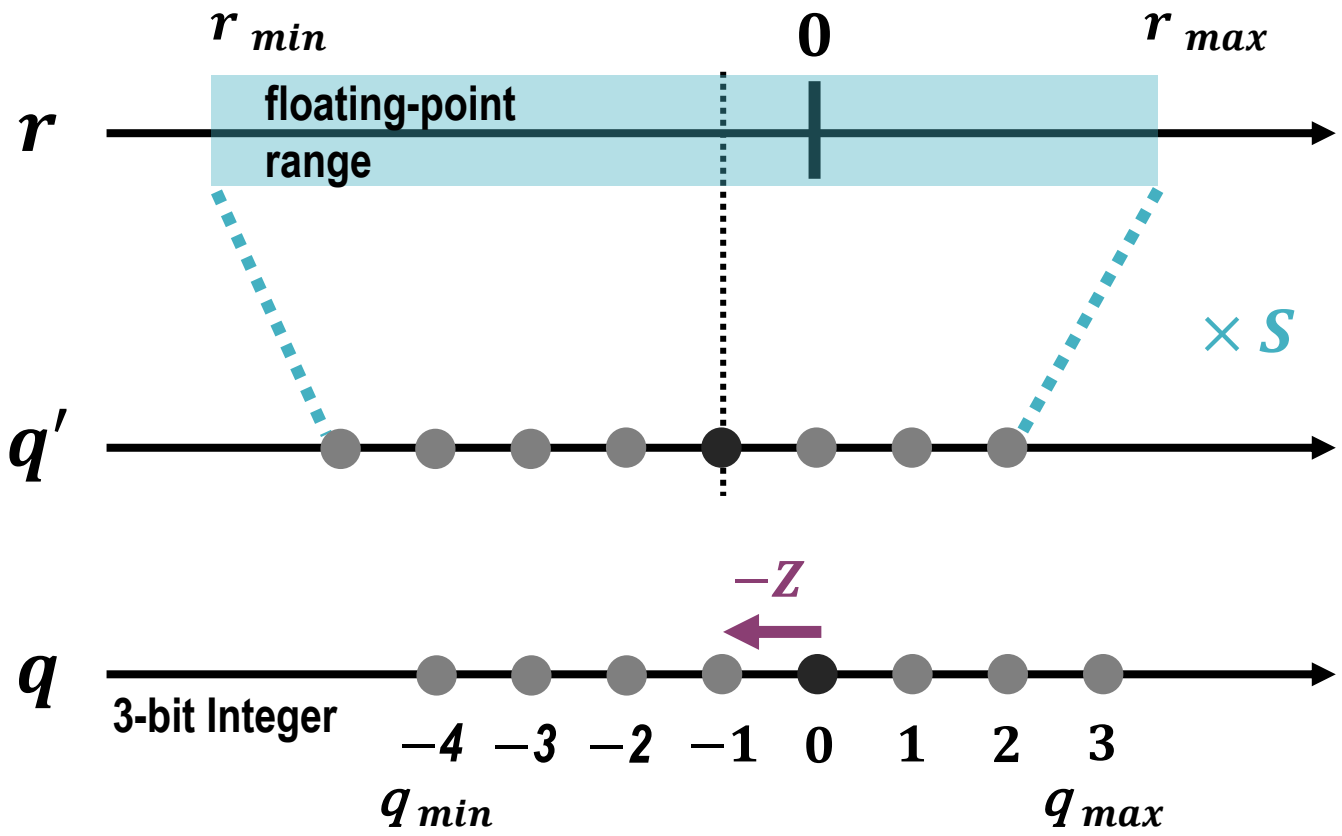
$$r = (q - Z) \times S$$

Floating-point | Integer | Integer | Floating-point



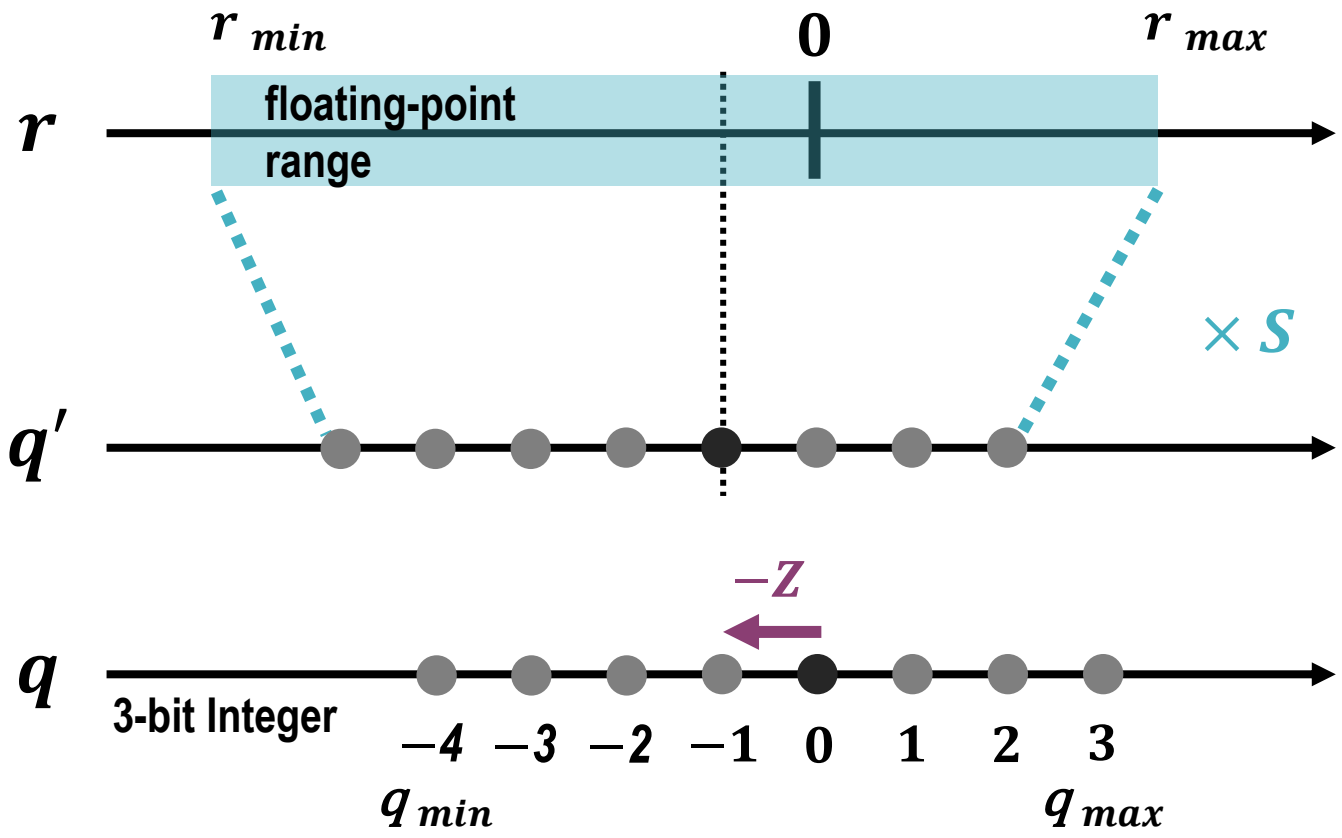
$$r = (q - Z) \times S$$

Floating-point | Integer | Integer | Floating-point



$$r = (q - Z) \times S$$

Floating-point | Integer | Integer | Floating-point



Calculate Scale S:

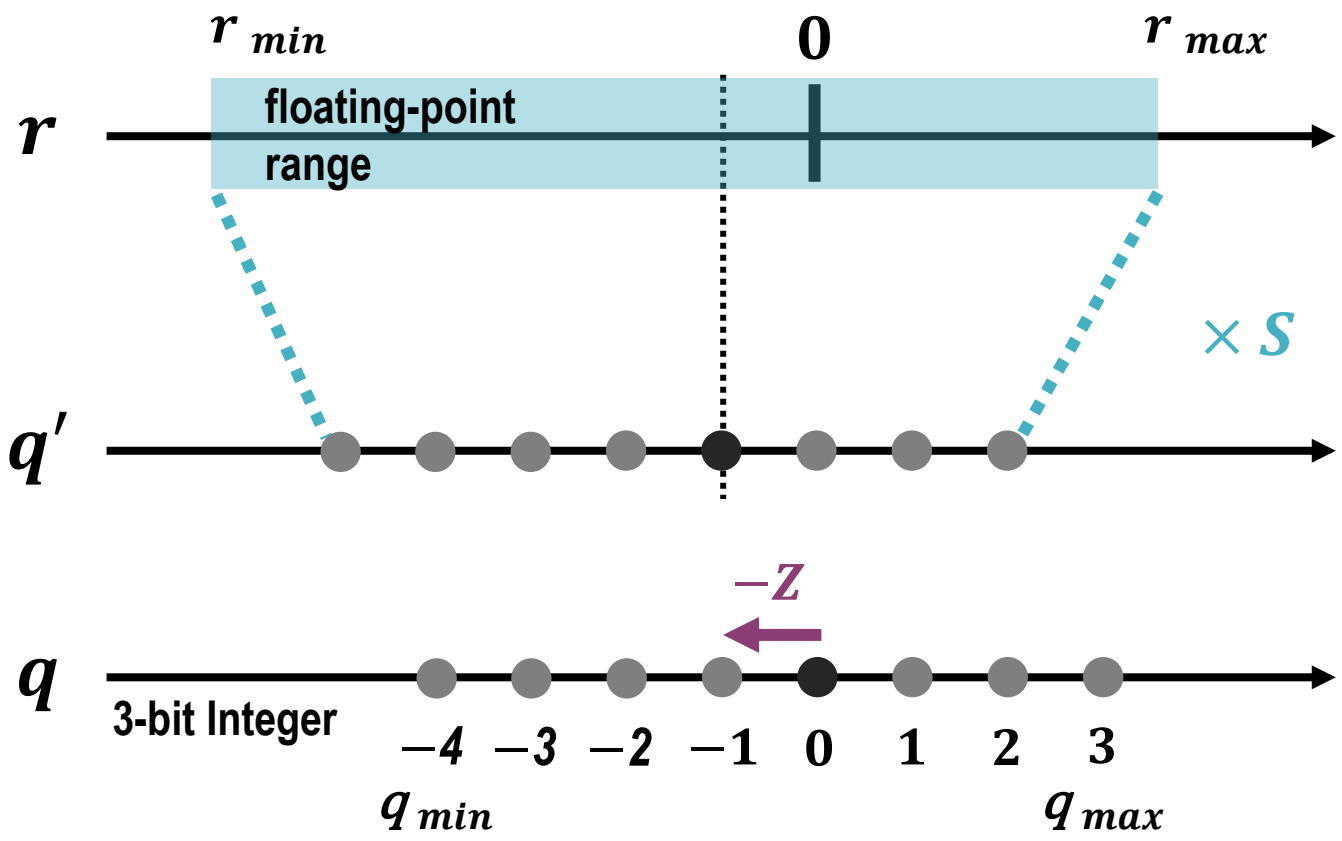
$$r = f(q)$$

$$\ominus \begin{cases} r_{max} = (q_{max} - Z) \times S \\ r_{min} = (q_{min} - Z) \times S \end{cases}$$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$r = (q - Z) \times S$$

Floating-point | Integer | Integer | Floating-point



Calculate Scale S:

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

Calculate Zero Point Z:

$$r_{min} = (q_{min} - Z) \times S$$

$$Z = q_{min} - \frac{r_{min}}{S}$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$

Linear Quantization

Quantize

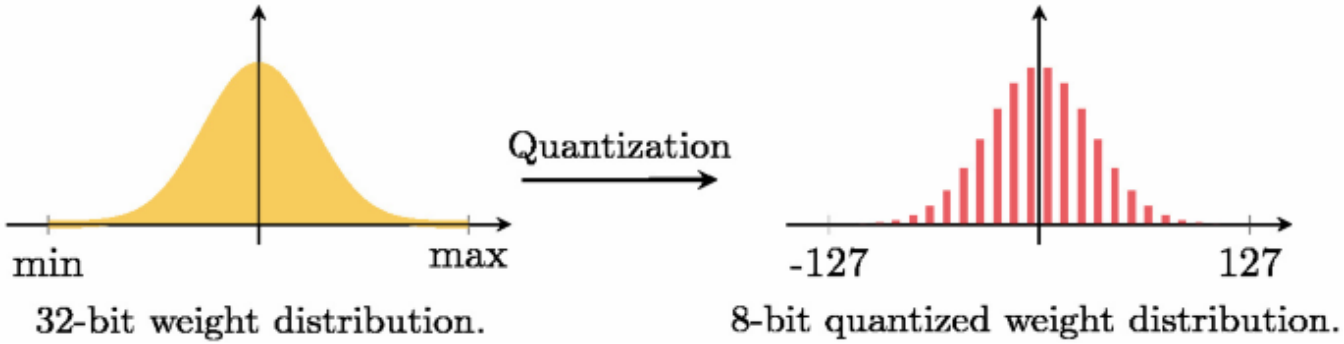
- $q = round(r \div S + Z)$

Dequantize

- $r = (q - Z) \times S$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$



32-bit weight distribution.

8-bit quantized weight distribution.

Try to quantize the following matrix to 2-bit int:

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

$r_{min} =$ $q_{min} =$

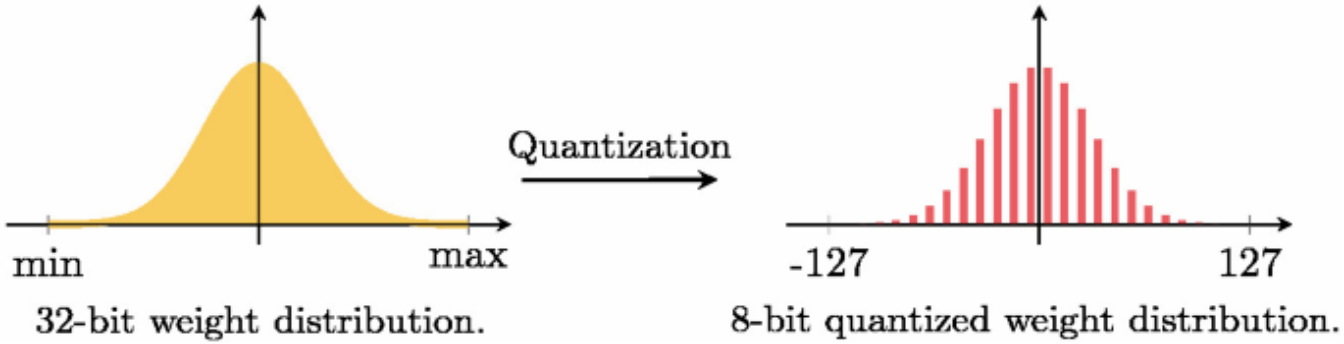
$r_{max} =$ $q_{max} =$

$S =$

$Z =$

Linear Quantization

- **Quantize**
  - $q = round(r \div S + Z)$
- **Dequantize**
  - $r = (q - Z) \times S$



$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$

Try to quantize the following matrix to 2-bit int:

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

$r_{min} = -1.09 \qquad q_{min} = -2$

$r_{max} = 2.12 \qquad q_{max} = 1$

$$S = \frac{2.12 - (-1.09)}{1 - (-2)} = 1.07$$

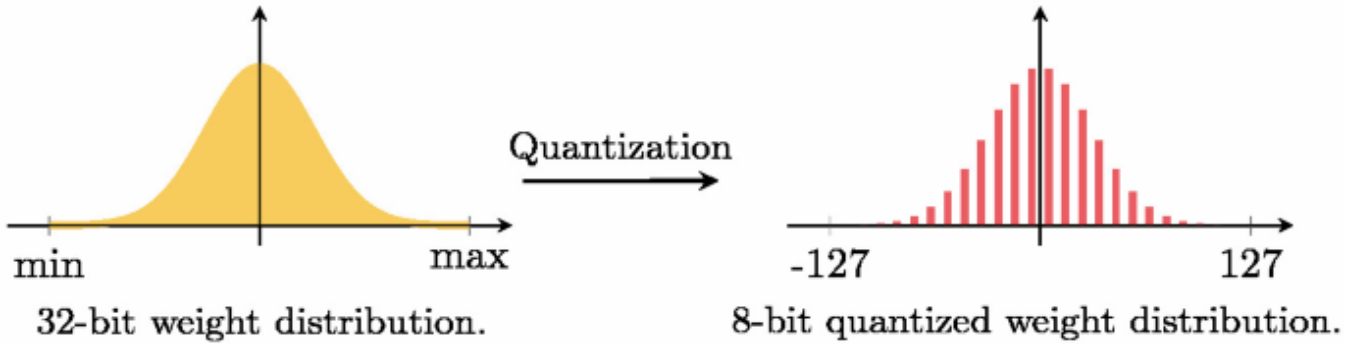
$$Z = round\left(-2 - \frac{-1.09}{1.07}\right) = -1$$

Linear Quantization

- **Quantize**
  - $q = round(r \div S + Z)$
- **Dequantize**
  - $r = (q - Z) \times S$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$Z = round\left(q_{min} - \frac{r_{min}}{S}\right)$$



32-bit weight distribution. 8-bit quantized weight distribution.

Try to quantize the following matrix to 2-bit int:

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

quantize

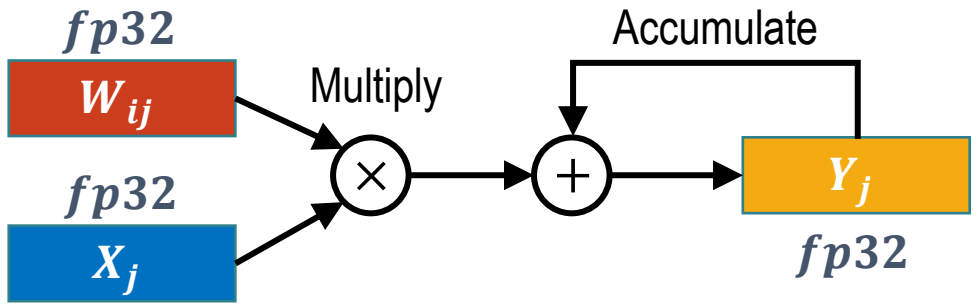
$S = 1.07$

$Z = -1$

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

Integer-Arithmetic-Only Quantization

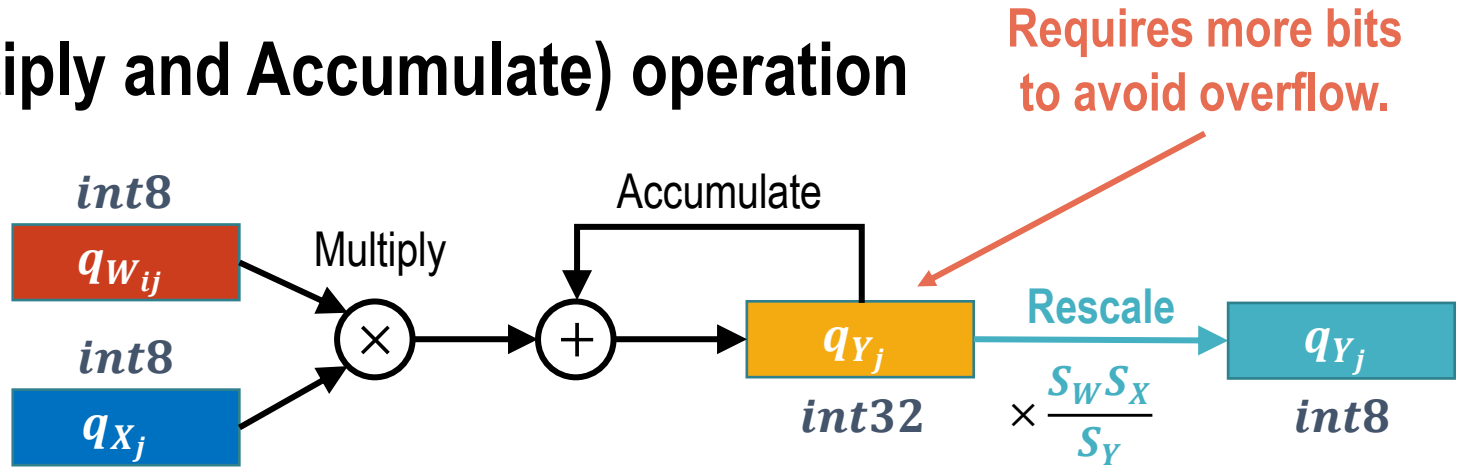
- **MAC (Multiply and Accumulate) operation**



$$Y = WX$$

Integer-Arithmetic-Only Quantization

- MAC (Multiply and Accumulate) operation



$$Y = WX$$

$$S_Y\,(q_Y - Z_Y) = S_W(q_W - Z_W) \cdot S_X(q_X - Z_X)$$

$$q_Y = \frac{S_W S_X}{S_Y}(q_W - Z_W)(q_X - Z_X) + Z_Y$$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

- Integer-Arithmetic-Only Inference

$$\boldsymbol{Y = WX}$$

$$\underbrace{q_Y}_{int8} = \underbrace{\frac{S_W S_X}{S_Y}}_{fp32}(q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X) + Z_Y$$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Integer-Arithmetic-Only Inference

$$\boldsymbol{Y = WX}$$

Dyadic Scaling:

$$\frac{S_W S_X}{S_Y} \cong 2^{-n} {M_0}^{int}$$

$$\overset{int8}{q_Y} = round\left(\overset{fp32}{\frac{S_W S_X}{S_Y}}(q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X)\right) + Z_Y$$

Can we avoid the floating-point operation here?

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Integer-Arithmetic-Only Inference

Dyadic Scaling:

$$\frac{S_W S_X}{S_Y} \cong 2^{-n} {M_0}^{int}$$

$$\boldsymbol{Y = WX}$$

$$\boldsymbol{q_Y} = \boldsymbol{round}\left(\frac{\boldsymbol{S_W S_X}}{\boldsymbol{S_Y}}(\boldsymbol{q_W q_X} - \boldsymbol{Z_W q_X} - \boldsymbol{Z_X q_W} + \boldsymbol{Z_W Z_X})\right) + \boldsymbol{Z_Y}$$

($\boldsymbol{int8}$ / $\boldsymbol{fp32}$)

$$\boldsymbol{q_Y} = \boldsymbol{round}\left(\boldsymbol{2^{-n} M_0} \cdot (\boldsymbol{q_W q_X} - \boldsymbol{Z_W q_X} - \boldsymbol{Z_X q_W} + \boldsymbol{Z_W Z_X})\right) + \boldsymbol{Z_Y}$$

Round to nearest bit-shift

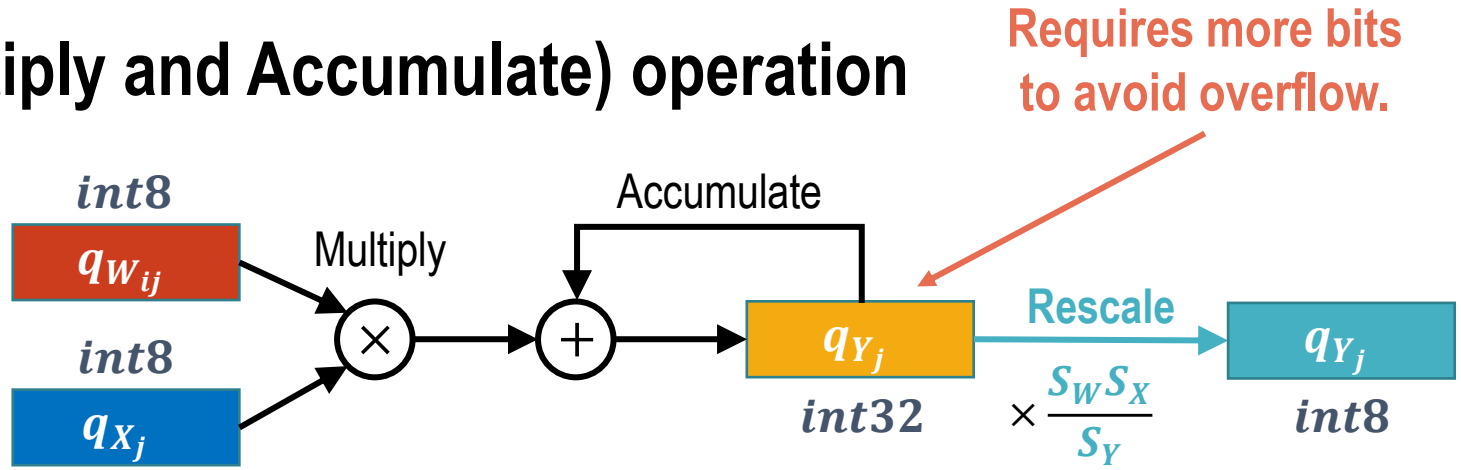
$$\boldsymbol{q_Y} = \left(\boldsymbol{M_0} \cdot (\boldsymbol{q_W q_X} - \boldsymbol{Z_W q_X} - \boldsymbol{Z_X q_W} + \boldsymbol{Z_W Z_X})\right) \gg \boldsymbol{n} + \boldsymbol{Z_Y}$$



Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Integer-Arithmetic-Only Quantization

- MAC (Multiply and Accumulate) operation



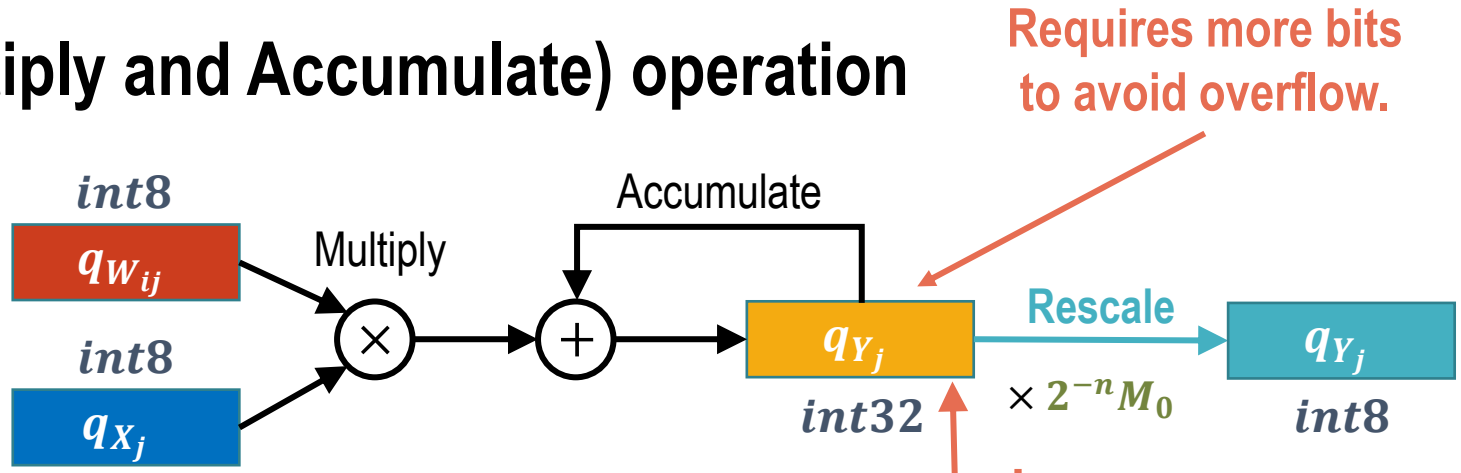
$$Y = WX$$

$$q_Y = \frac{S_W S_X}{S_Y}(q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X) + Z_Y$$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Integer-Arithmetic-Only Quantization

- MAC (Multiply and Accumulate) operation



$$Y = WX$$

$$q_Y = \left(M_0 \cdot \left(q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X\right)\right) \gg n + Z_Y$$

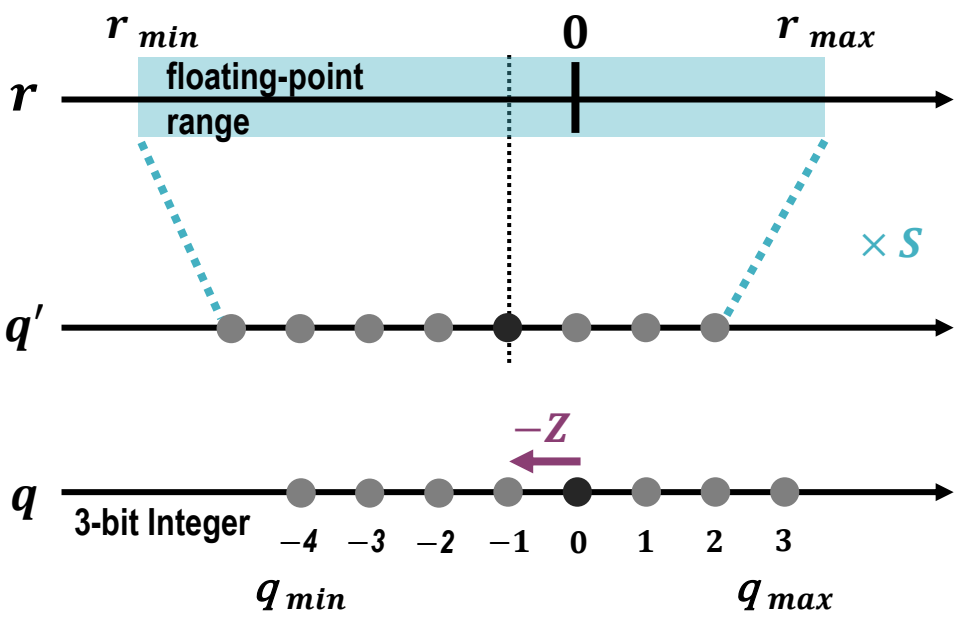
precompute,
can be fused into bias term

$$Z_W = 0?$$

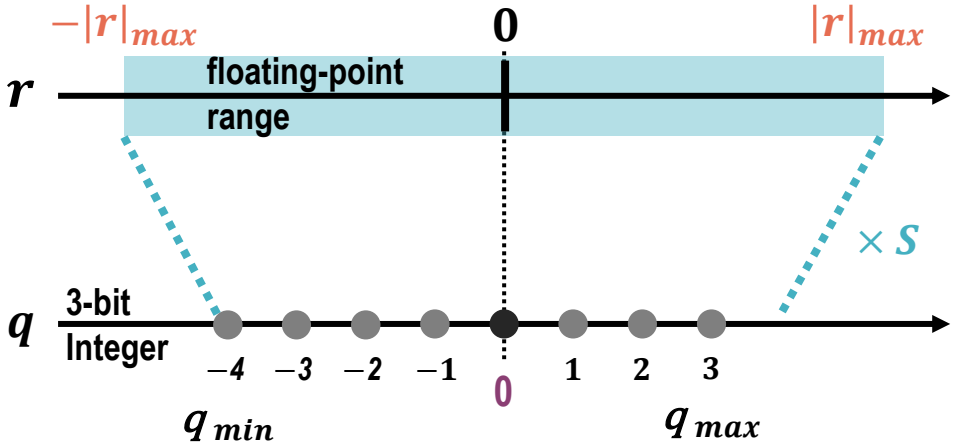
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Symmetric Linear Quantization

- **Asymmetric ($Z \neq 0$) vs. Symmetric ($Z = 0$)**



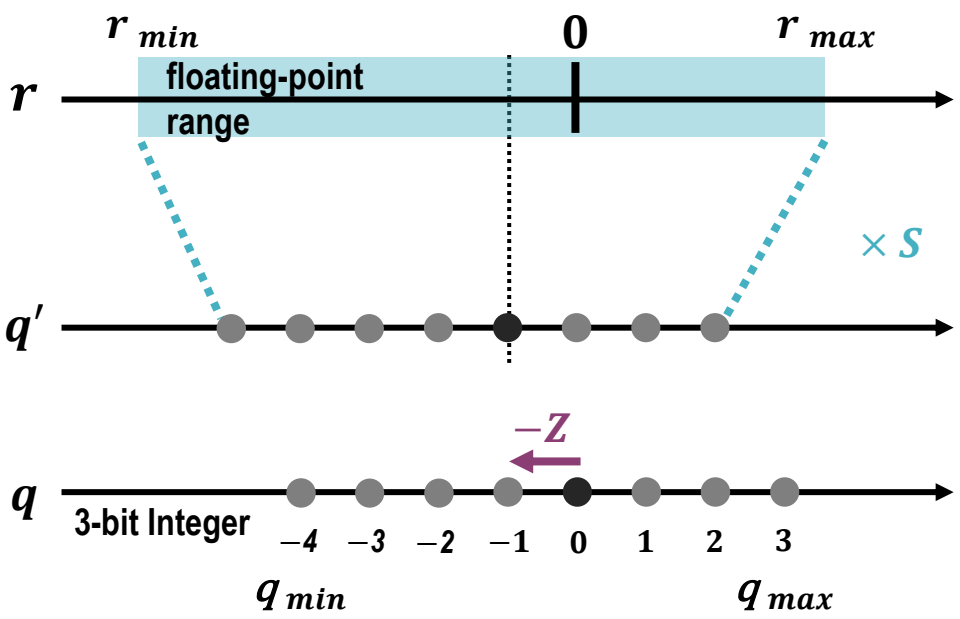
Asymmetric ($Z \neq 0$)



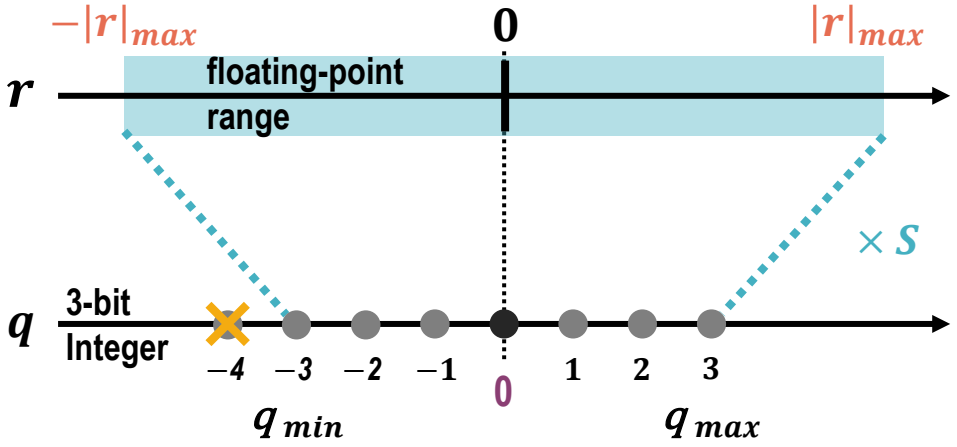
Symmetric ($Z = 0$)

Symmetric Linear Quantization

- **Asymmetric ($Z \neq 0$) vs. Symmetric ($Z = 0$)**



Asymmetric ($Z \neq 0$)



$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} = \frac{|r|_{max}}{q_{max}} = \frac{|r|_{max}}{2^{N-1} - 1}$$

Symmetric ($Z = 0$)

Integer-Arithmetic-Only Inference

So far, we have ignored the bias term.
Now we consider the following fully-connected layer with bias.

$$\boldsymbol{Y = WX + b}$$

$$\boldsymbol{S_Y\,(q_Y - Z_Y) = S_W(q_W - Z_W)\cdot S_X(q_X - Z_X) + S_b(q_b - Z_b)}$$

$$\downarrow \quad \boldsymbol{Z_W = 0}$$

$$\boldsymbol{S_Y\,(q_Y - Z_Y) = S_W S_X(q_W q_X - Z_X q_W) + S_b(q_b - Z_b)}$$

$$\downarrow \quad \boldsymbol{Z_b = 0,\quad S_b = S_W S_X}$$

$$\boldsymbol{S_Y\,(q_Y - Z_Y) = S_W S_X(q_W q_X - Z_X q_W + q_b)}$$

$$\boldsymbol{q_Y = \frac{S_W S_X}{S_Y}(q_W q_X - Z_X q_W + q_b) + Z_Y}$$

Integer-Arithmetic-Only Inference

So far, we have ignored the bias term.
Now we consider the following fully-connected layer with bias.

$$\boldsymbol{Y = WX + b}$$

$$\downarrow \quad \boldsymbol{Z_W = 0\,,\ \ Z_b = 0,\ \ \ S_b = S_W S_X}$$

$$\boldsymbol{q_Y = \frac{S_W S_X}{S_Y}(q_W q_X} \underbrace{\boldsymbol{- Z_X q_W + q_b}}_{\text{precompute}}\boldsymbol{) + Z_Y}$$

$$\downarrow \quad \boldsymbol{q_{bias} = q_b - Z_X q_W}$$

$$\boldsymbol{q_Y = \frac{S_W S_X}{S_Y}(q_W q_X + q_{bias}) + Z_Y}$$

$$\downarrow \quad \boldsymbol{2^{-n} {M_0}^{int} \cong \frac{S_W S_X}{S_Y}}$$

$$\boldsymbol{q_Y = \left(M_0 \cdot (q_W q_X + q_{bias})\right) \gg n + Z_Y}$$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Linear Quantized Fully-Connected Layer

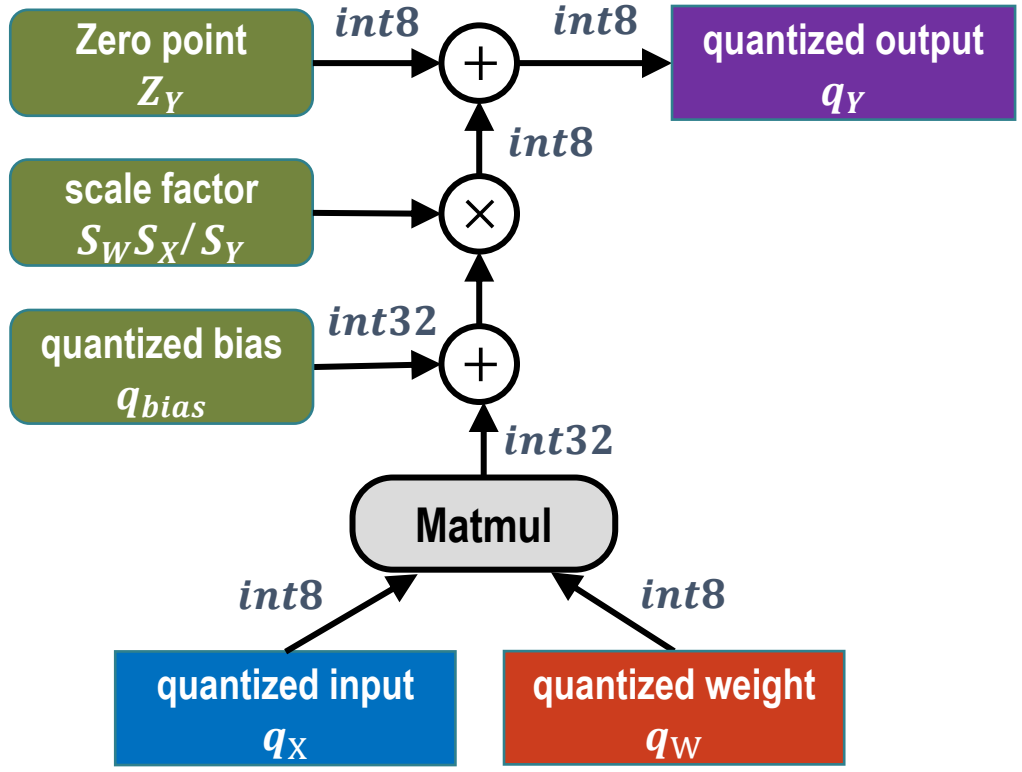
- $Y = WX + \mathbf{b}$
  - **Weight:** symmetric quant
  - **Activation:** asymmetric quant

- $Z_W = 0$
- $Z_b = 0, \quad S_b = S_W S_X$
- $q_{bias} = q_b - Z_X q_W$
- $2^{-n} M_0{}^{int} \cong \frac{S_W S_X}{S_Y}$

8-bit Int Mult. 32-bit Int Add. 8-bit Int Add.

$$q_Y = \left(M_0 \cdot (q_W q_X + q_{bias}) \right) \gg n + Z_Y$$

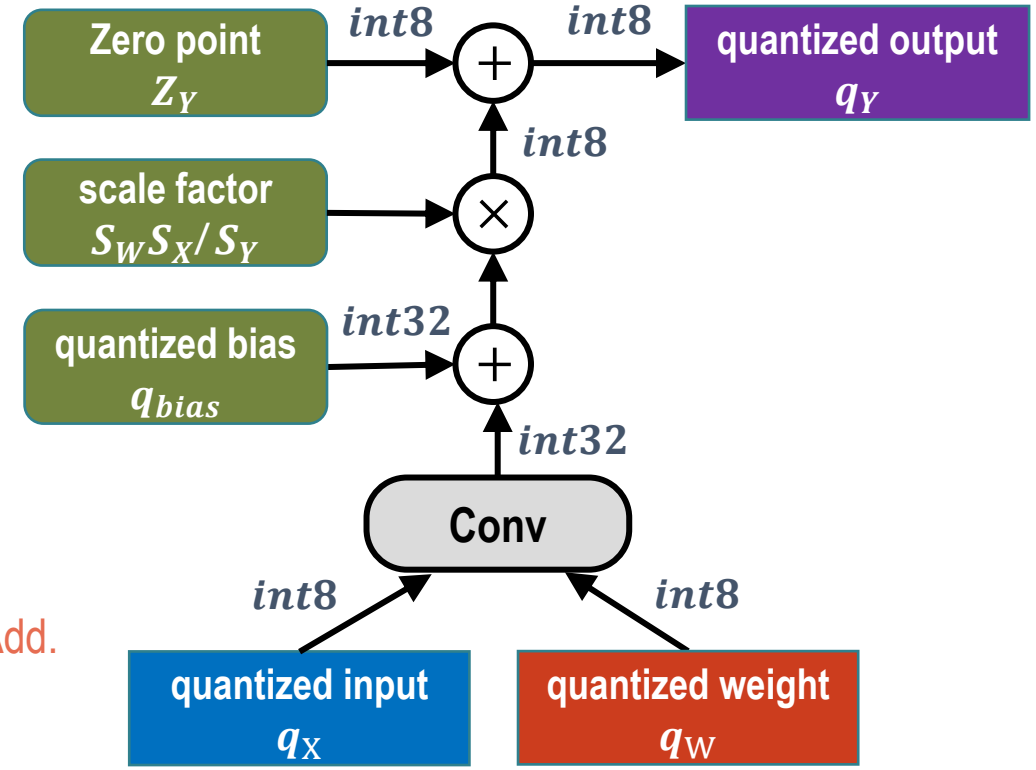
8-bit Int Matmul. Rounding bit-shift



Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Linear Quantized Convolution Layer

- $Y = Conv(W, X) + \mathbf{b}$
  - $Z_W = 0$
  - $Z_b = 0, \quad S_b = S_W S_X$
  - $q_{bias} = q_b - Conv(q_W, Z_X)$
  - $2^{-n} {M_0}^{int} \cong \frac{S_W S_X}{S_Y}$



8-bit Int Mult. 32-bit Int Add. 8-bit Int Add.

$$q_Y = \left(M_0 \cdot (Conv(q_W, q_X) + q_{bias})\right) \gg n + Z_Y$$

8-bit Int Conv. Rounding bit-shift

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Outline

- Post-Training Quantization (PTQ)
- Quantize-Aware Training (QAT)
- Quantizing Large Models



Post-Training Quantization (PTQ)

How do we obtain optimal linear quantization parameters (S, Z) ?



Post-Training Quantization (PTQ)

- Quantization Granularity
- Dynamic Range Clipping
- Rescaling
- Rounding

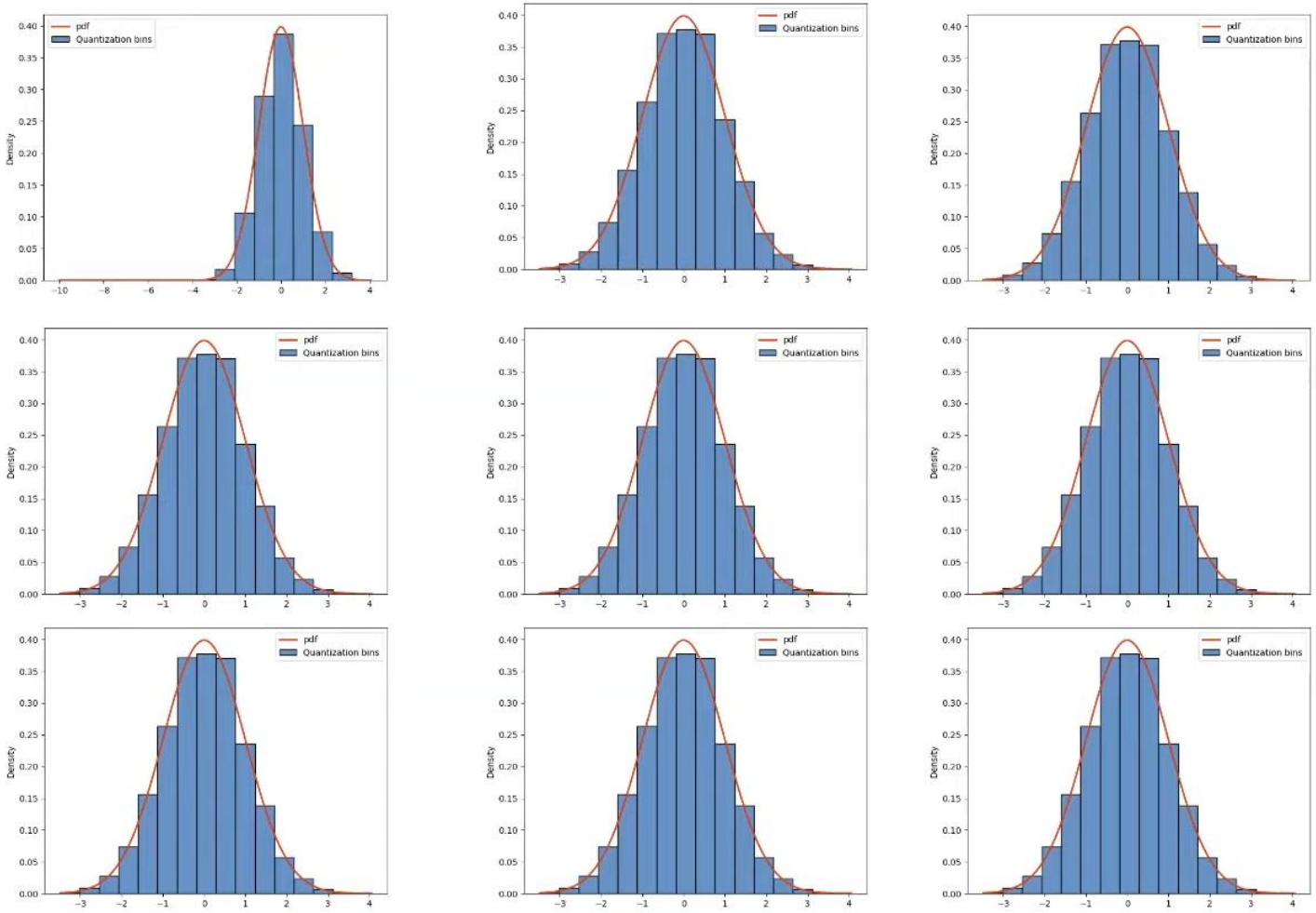


Post-Training Quantization (PTQ)

- **Quantization Granularity**
- Dynamic Range Clipping
- Rescaling
- Rounding

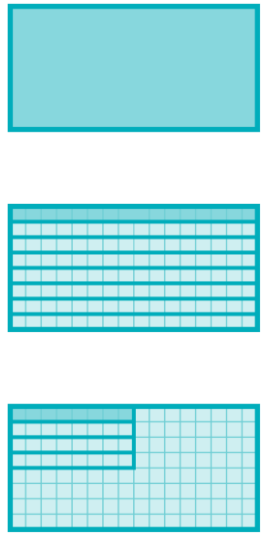


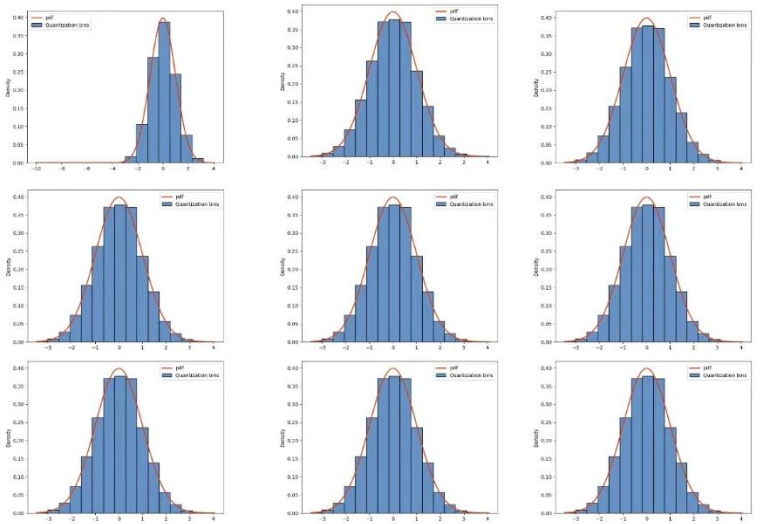
Quantization Granularity



Quantization Granularity

- **Per-tensor(per-layer) Quantization**
- **Per-channel Quantization**
- **Group-wise Quantization**
  - Per-Vector Quantization

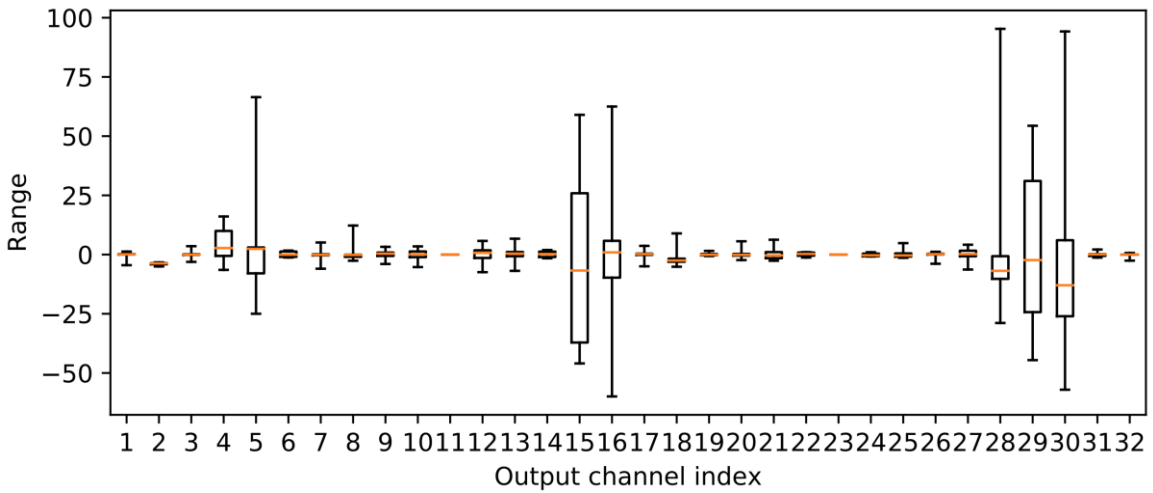




Per-Tensor Weight Quantization

Symmetric Linear Quantization on Weights

$$|r|_{max} = |W|_{max} \qquad S = \frac{|r|_{max}}{q_{max}}$$



Per-Tensor Quantization

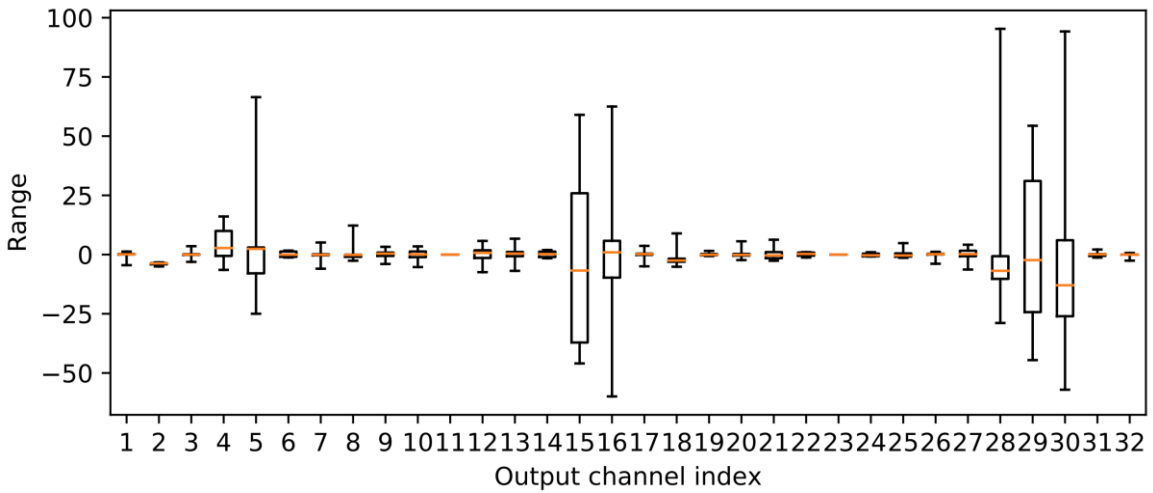
- Applies a uniform scale S across the entire weight tensor
  - Works well for larger models
  - Accuracy drops for small models
- Large differences (more than 100x) in ranges of different channels
  - ➢ **Outliers**

Data-Free Quantization Through Weight Equalization and Bias Correction [Nagel et al., ICCV 2019]

Per-Tensor Weight Quantization

Symmetric Linear Quantization on Weights

$$|r|_{max} = |W|_{max} \qquad S = \frac{|r|_{max}}{q_{max}}$$



Per-Tensor Quantization

- Applies a uniform scale S across the entire weight tensor
  - Works well for larger models
  - Accuracy drops for small models
- Large differences (more than 100x) in ranges of different channels
  - ➢ **Outliers**

How can we reduce the impact of an outlier value?

➢ Per-Channel Quantization



Data-Free Quantization Through Weight Equalization and Bias Correction [Nagel et al., ICCV 2019]

Per-Channel Weight Quantization

Example: 2-bit linear quantization

ic

oc				
	2.09	-0.98	1.48	0.09
	0.05	-0.14	-1.09	2.12
	-0.97	1.92	0	-1.03
	1.87	0	1.53	1.49

Per-Channel Quantization

Per-Tensor Quantization

$$|r|_{max} = 2.12$$

$$S = \frac{|r|_{max}}{q_{max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed



Per-Channel Weight Quantization

Example: 2-bit linear quantization

ic

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.09	2.12
-0.97	1.92	0	-1.03
1.87	0	1.53	1.49

oc

Per-Channel Quantization

$$|r|_{max} = 2.09 \quad S_0 = 2.09$$
$$|r|_{max} = 2.12 \quad S_1 = 2.12$$
$$|r|_{max} = 1.92 \quad S_2 = 1.92$$
$$|r|_{max} = 1.87 \quad S_3 = 1.87$$

1	0	1	0
0	0	-1	1
0	1	0	-1
1	0	1	1

Quantized

2.09	0	2.09	0
0	0	-2.12	2.12
0	1.92	0	-1.92
1.87	0	1.87	1.87

Reconstructed

$$\|W - Sq_W\|_F$$

<

Per-Tensor Quantization

$$|r|_{max} = 2.12$$

$$S = \frac{|r|_{max}}{q_{max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

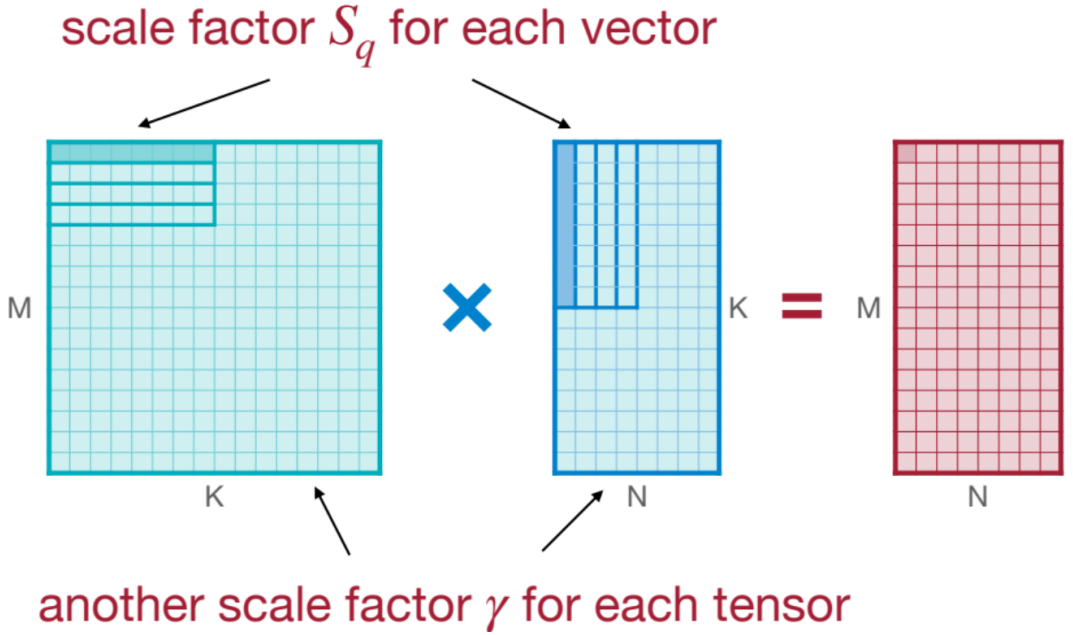
Reconstructed

$$\|W - Sq_W\|_F$$

VS-Quant: Per-vector Scaled Quantization

$$\boldsymbol{r = (q - Z) \cdot S \rightarrow r = (q - Z) \cdot S_q \cdot \gamma}$$

- S_q: Per-vector scale vector (integer)
- γ : Coarse grained scale factor (floating-point)
- Achieves a balance between accuracy and hardware efficiency by
  - less expensive INT scale at finer granularity
  - more expensive FP scale at coarser granularity



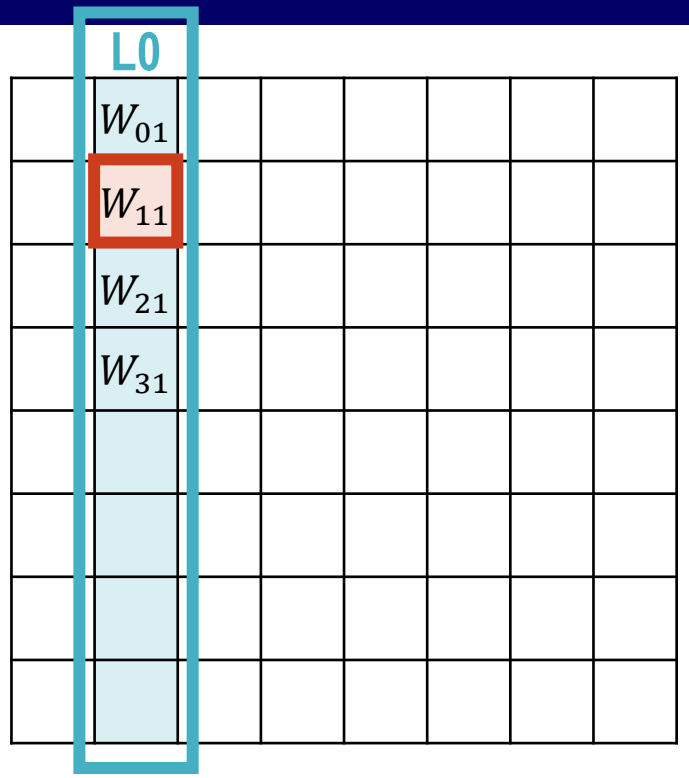
Memory overhead of two-level scaling:

Assume 4-bit quantization, 4-bit per-vector scale, group_size=16.

➢ The effective bit width is: 4 + 4/16 = 4.25 bits

VS-Quant: Per-Vector Scaled Quantization for Accurate Low-Precision Neural Network Inference [Steve Dai,et al.]

Group Quantization – Multi-level scaling scheme



$$r = (q - Z) \cdot S \rightarrow$$

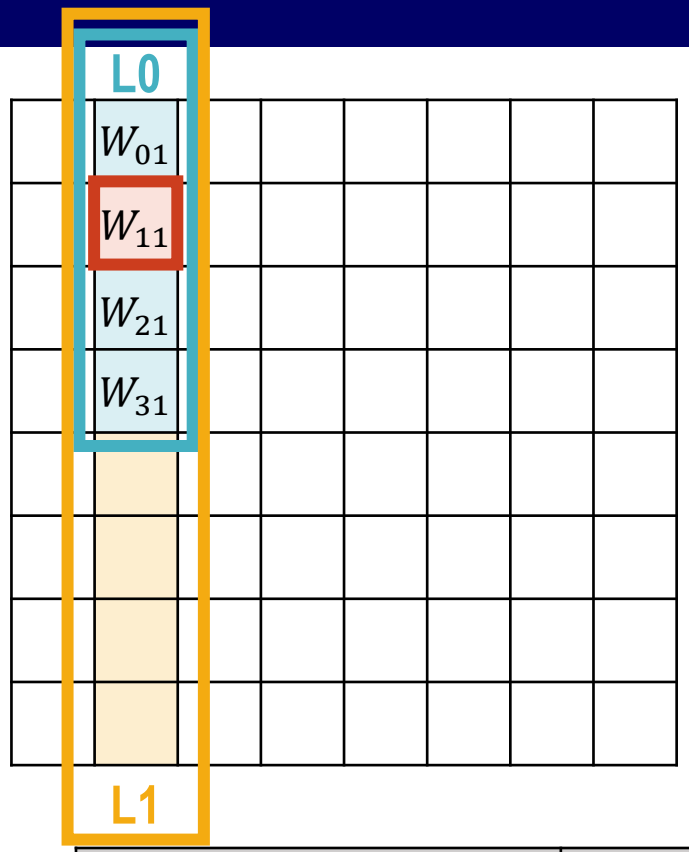
$$r = (q - Z) \cdot S_{l_0} \cdot S_{l_1} \cdot \cdots$$

FP16 s_{l_0} INT4 q

- r : real number value
- q : quantized value
- z : zero point (z=0 is symmetric quant)
- s : scale factors of different levels

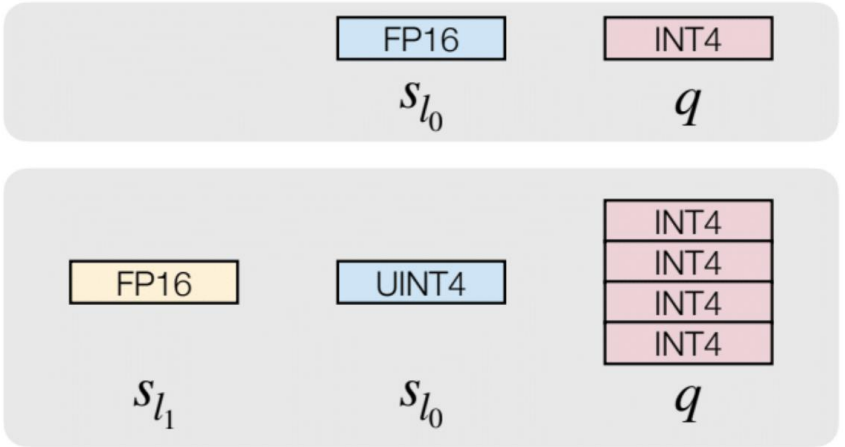
Method	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit-Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4

Group Quantization – Multi-level scaling scheme



$$r = (q - Z) \cdot S \rightarrow$$

$$r = (q - Z) \cdot S_{l_0} \cdot S_{l_1} \cdot \cdots$$



- r : real number value
- q : quantized value
- z : zero point (z=0 is symmetric quant)
- s : scale factors of different levels

Method	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit-Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4
VSQ	INT4	16	UINT4	Per Channel	FP16	4+4/16=4.25

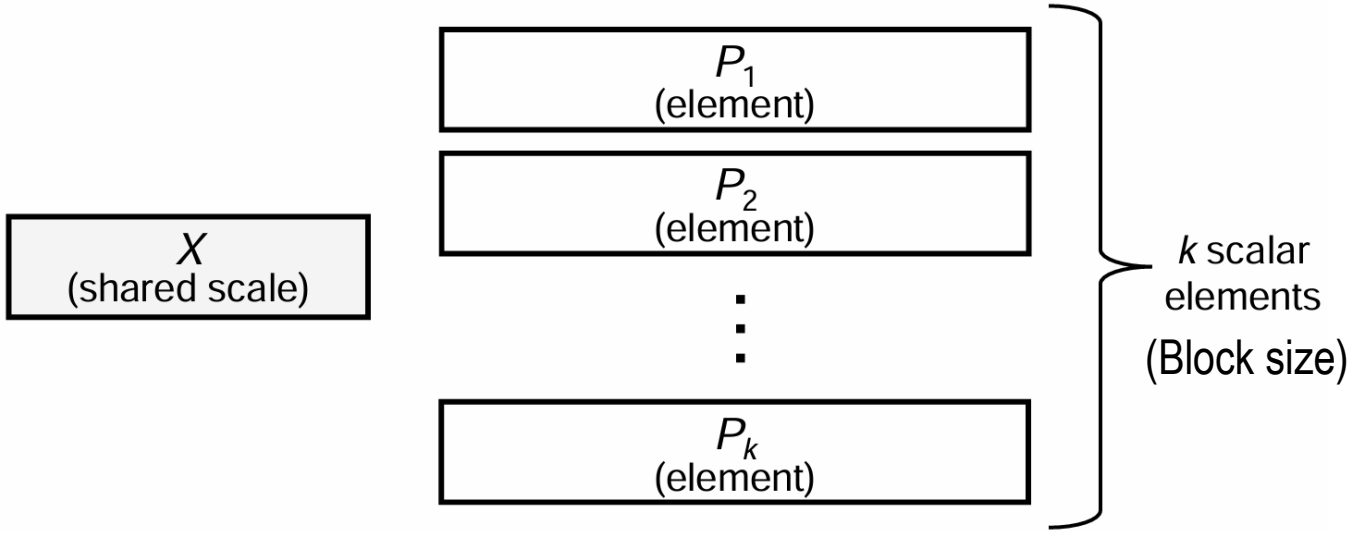
VS-Quant Accuracy Comparison

Bitwidths	$S_w = 3\ bit$ $S_a = 4\ bit$	$S_w = 4\ bit$ $S_a = 4\ bit$	$S_w = 6\ bit$ $S_a = 6\ bit$	$S = fp32$	*Best Per-channel*
Wt=4 Act=3U	73.64	73.53	74.69	74.71	67.20
Wt=4 Act=4U	73.39	74.36	75.35		70.76
Wt=4 Act=8U	73.65	74.66	75.42	75.42	72.30
Wt=8 Act=4U	74.48	75.08	75.96	75.91	75.11
Wt=8 Act=8U	74.61	75.15	76.10	76.15	76.16

ResNet50 on ImageNet with integer per-vector scale factors



Microscaling Formats (MX)



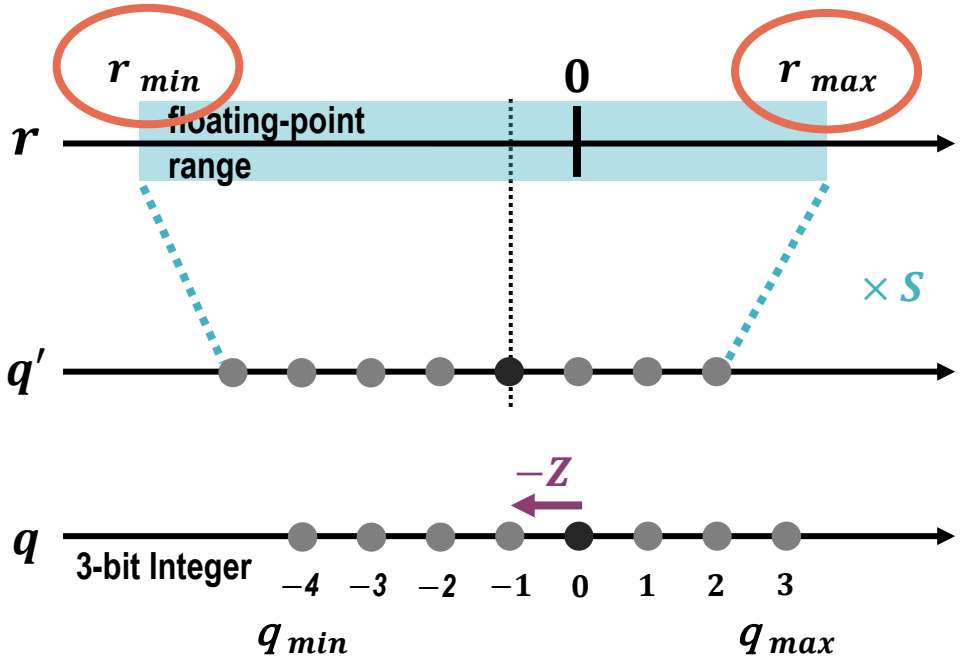
Format Name	Block Size	Scale Data Format	Scale Bits	Element Data Format	Element Bit-Width
MXFP8	32	E8M0	8	FP8 (E4M3 / E5M2)	8
MXFP6	32	E8M0	8	FP6 (E2M3 / E3M2)	6
MXFP6	32	E8M0	8	FP4 (E2M1)	4
MXINT8	32	E8M0	8	INT8	8

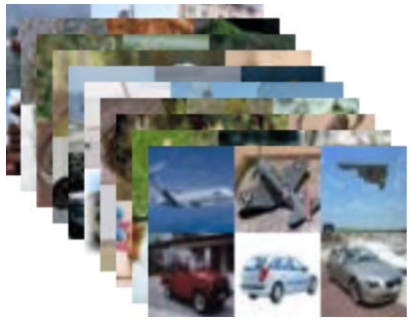
Post-Training Quantization (PTQ)

- Quantization Granularity
- **Dynamic Range Clipping**
- Rescaling
- Rounding

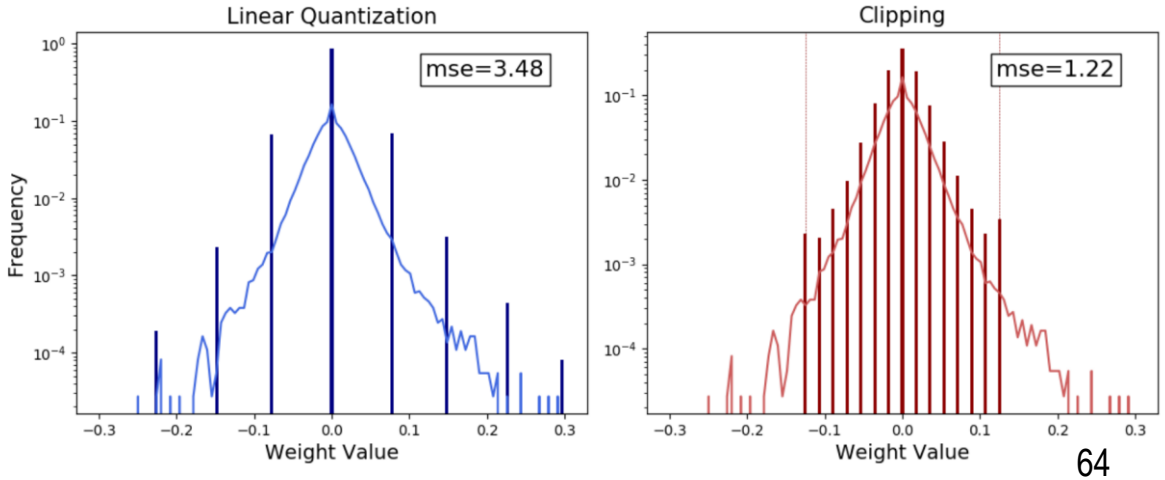


Linear Quantization on Activations



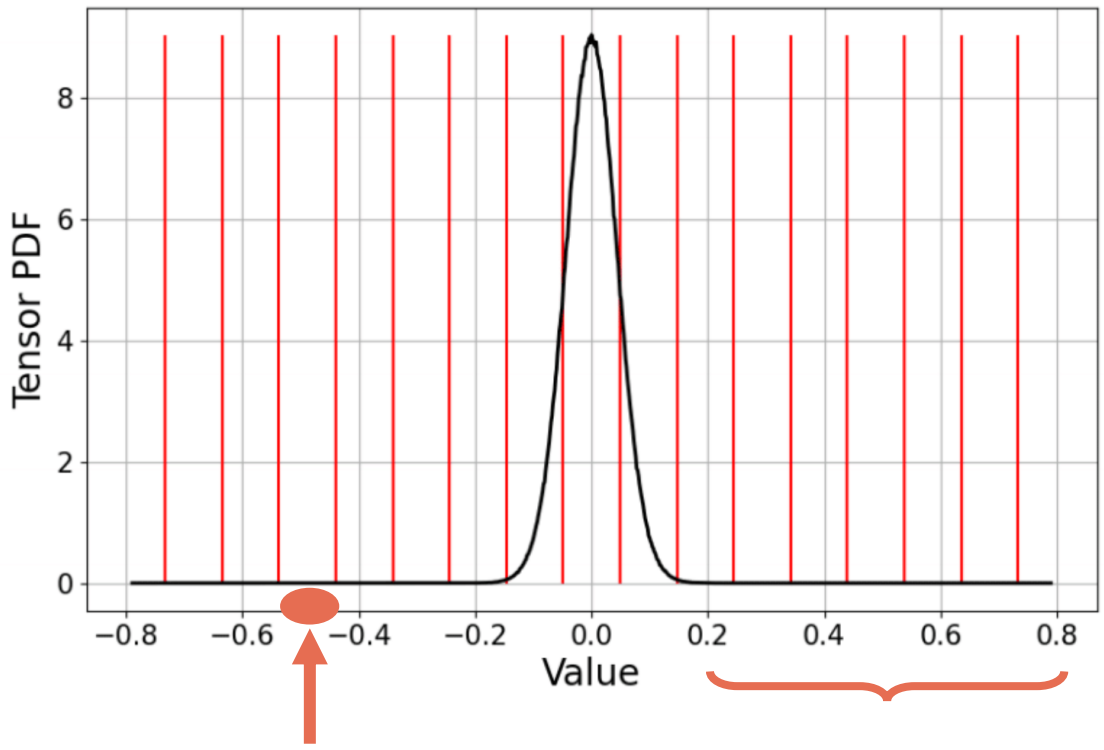


- Unlike weights, activation ranges vary by input.
- **Before** deploying the model, we need to first collect activation statistics to determine the average range.
- Values higher than the specified range will be clipped.

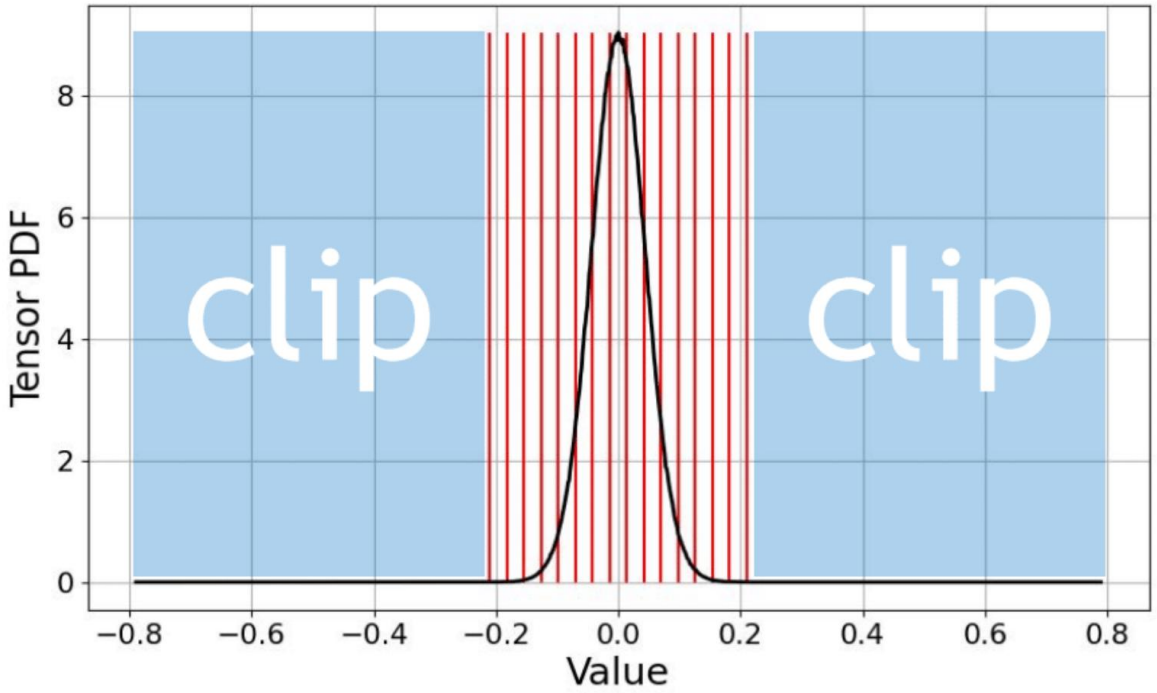


Dynamic Range for Activation Quantization

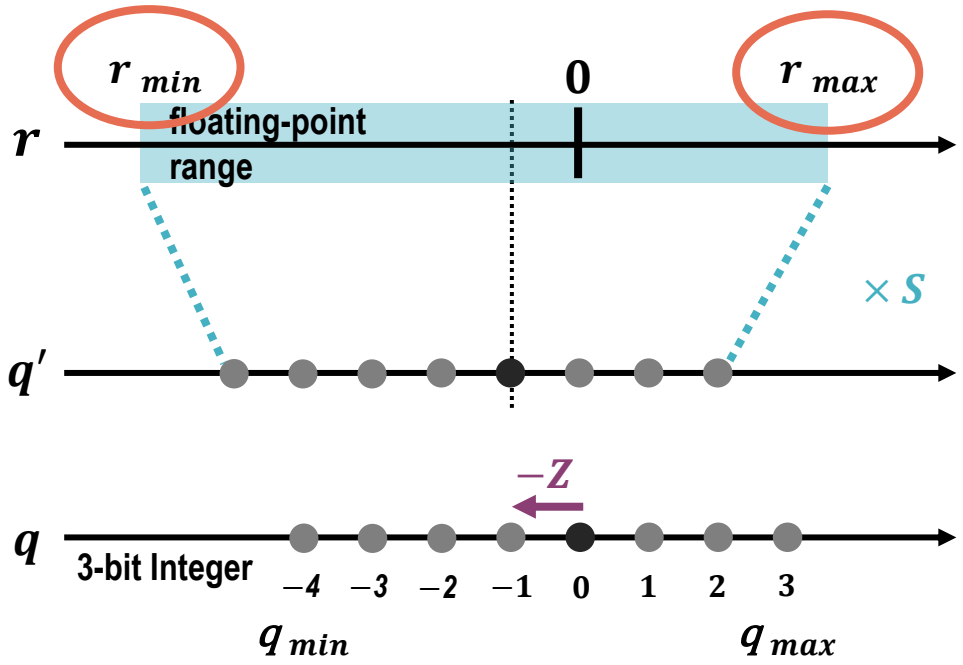
- **Clipping (or Clamping)**



- Reduces quantization noise
- Increases outliers' error



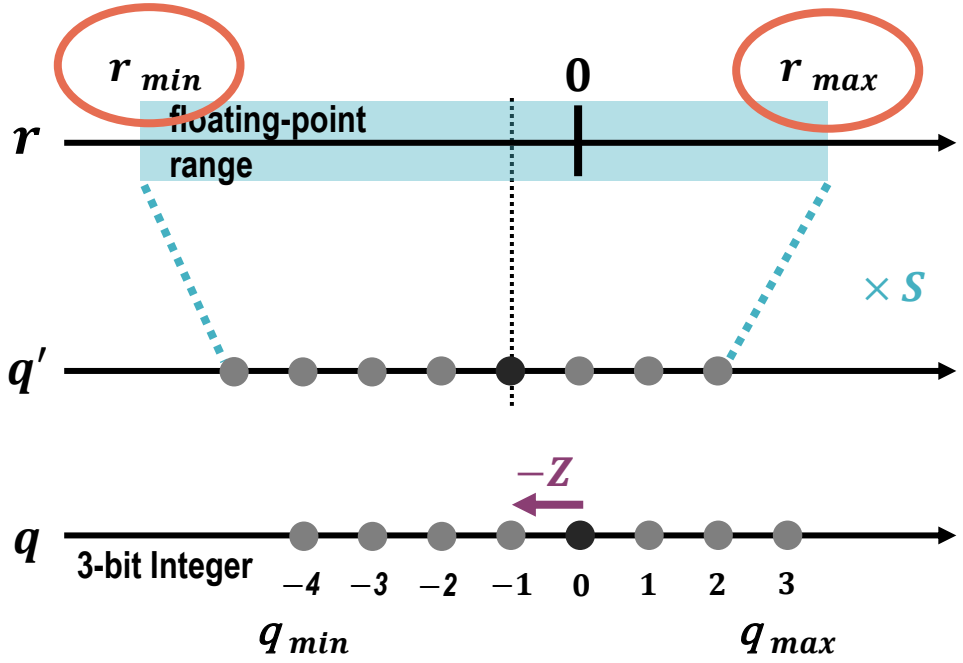
Dynamic Range for Activation Quantization



- **Min-max**
  - Obtain ranges by finding the min/max value of the tensor

Dynamic Range for Activation Quantization

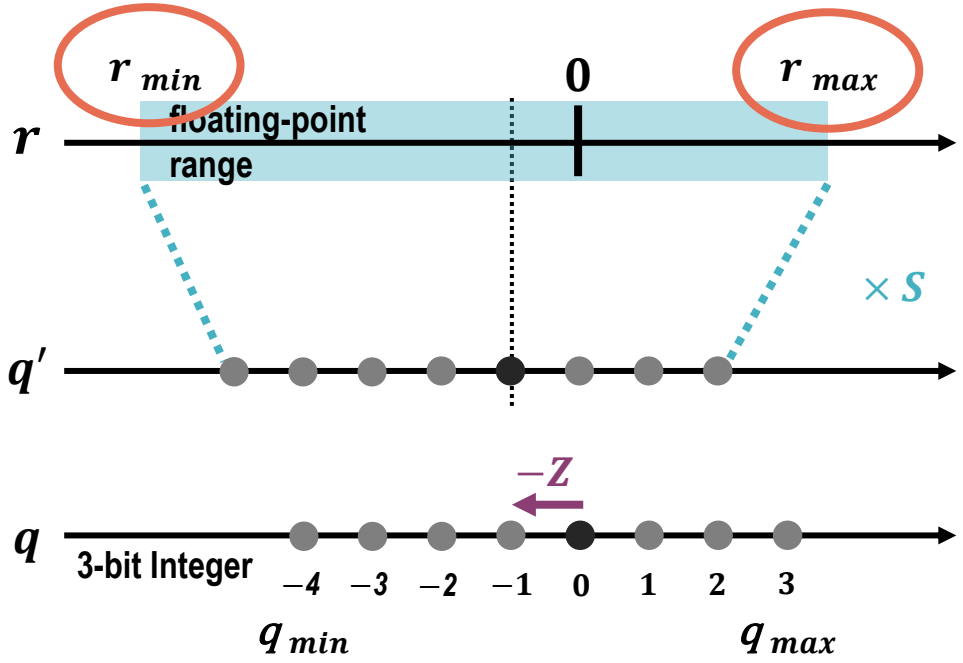
$$(\hat{r}_{min}, \hat{r}_{max})^t = \alpha \cdot (r_{min}, r_{max})^t + (1-\alpha) \cdot (\hat{r}_{min}, \hat{r}_{max})^{t-1}$$



- **Min-max**
  - Obtain ranges by finding the min/max value of the tensor
- **Exponential Moving Average (EMA)**
  - Smoothed version of **min-max**
  - Observed ranges are smoothed across hundreds of calibration steps.

Dynamic Range for Activation Quantization

$$(\hat{r}_{min}, \hat{r}_{max})^t = \alpha \cdot (r_{min}, r_{max})^t + (1-\alpha) \cdot (\hat{r}_{min}, \hat{r}_{max})^{t-1}$$



- **Min-max**
  - Obtain ranges by finding the min/max value of the tensor
- **Exponential Moving Average (EMA)**
  - Smoothed version of **min-max**
  - Observed ranges are smoothed across hundreds of calibration steps.
- **Mean-Square-Error (MSE)**
  - Find min/max value that minimize the MSE between the original and the quantized tensor:

$$\underset{q_{min}, q_{max}}{\operatorname{argmin}} \left\| W - \widetilde{W}(q_{min}, q_{max}) \right\|_F^2$$

MSE vs. min-max Comparison

- **MSE outperforms min-max on low bit-widths**

Model (FP32 accuracy)	ResNet18 (69.68)			MobileNetV2 (71.72)		
Bit-width	W8	W6	W4	W8	W6	W4
Min-Max	**69.57**	63.90	0.12	**71.16**	64.48	0.59
MSE	69.45	**64.64**	**18.82**	71.15	**65.43**	**13.77**
Min-Max (Per-channel)	69.60	69.08	44.49	71.21	68.52	18.40
MSE (Per-channel)	**69.66**	**69.24**	**54.67**	**71.46**	**68.89**	**27.17**



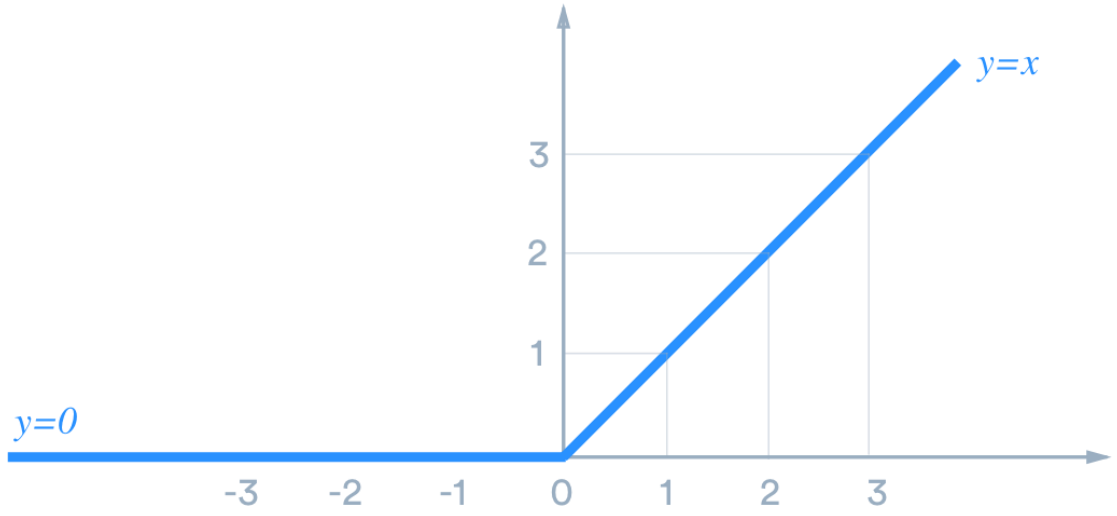
Post-Training Quantization (PTQ)

- Quantization Granularity
- Dynamic Range Clipping
- **Rescaling**
- Rounding



Cross-Layer Equalization (CLE)

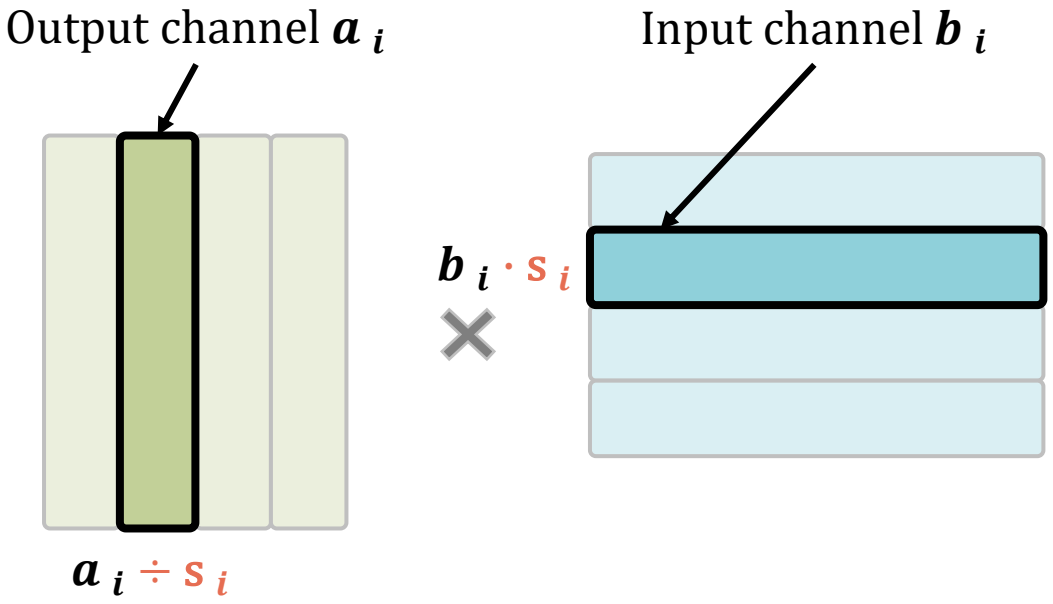
- **Rescale adjacent weight ranges in advance without affecting the output.**



$$relu(x) = max(0, x)$$

We have that

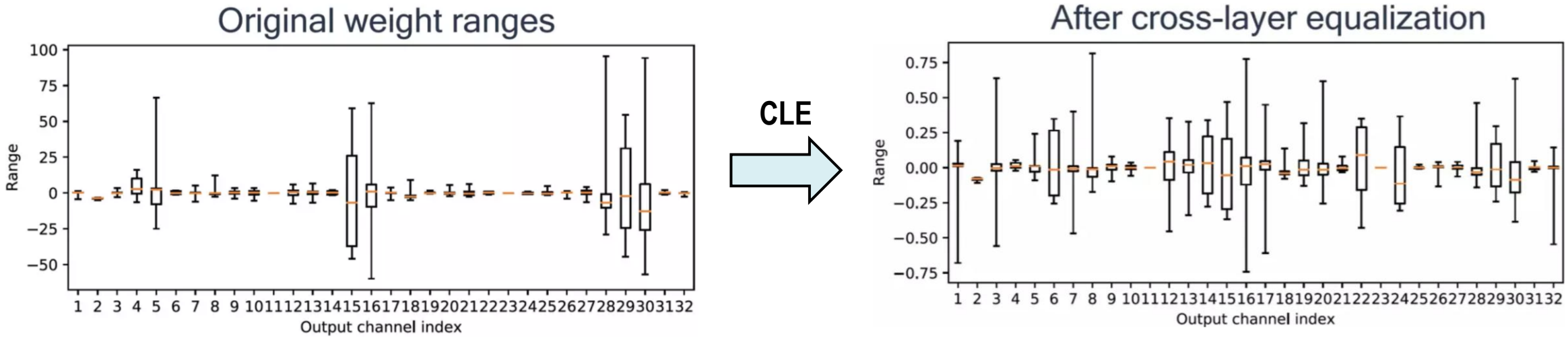
$$relu(\boldsymbol{s}x) = \boldsymbol{s} \cdot relu(x)$$



Data-Free Quantization Through Weight Equalization and Bias Correction [Nagel et al., ICCV 2019]

Cross-Layer Equalization (CLE)

- **Apply CLE to the weights before the quantization process.**

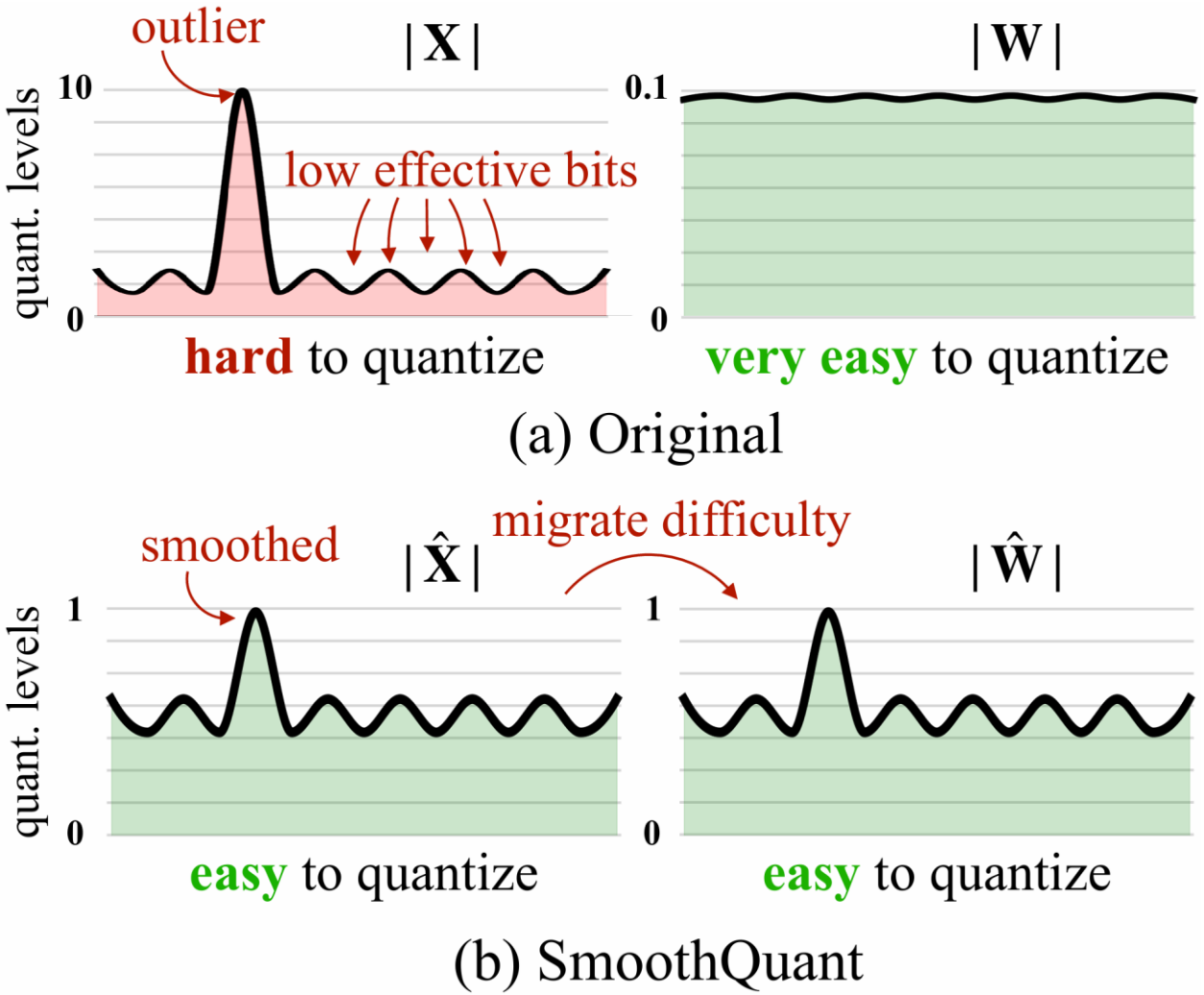


Method	Top-1 Accuracy (INT8)
Per-Tensor	0.1
Per-Channel	69.7
DFQ (per-tensor)	71.2

Current method

MobileNetV2 results for ImageNet ($fp32\ acc$: 71.71)

SmoothQuant



$$Y = XW$$

$\times 0.1$ (down) on X; $\times 10$ (up) on W

- Activation is harder to quantize due to outliers
- Migrate the quantization difficulty from activation to weight

OPT-IML-30B	LAMBADA ↑	WikiText ↓
FP16	69.12%	14.26
W8A8	4.21%	576.53
SmoothQuant	69.77%	14.37

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

Post-Training Quantization (PTQ)

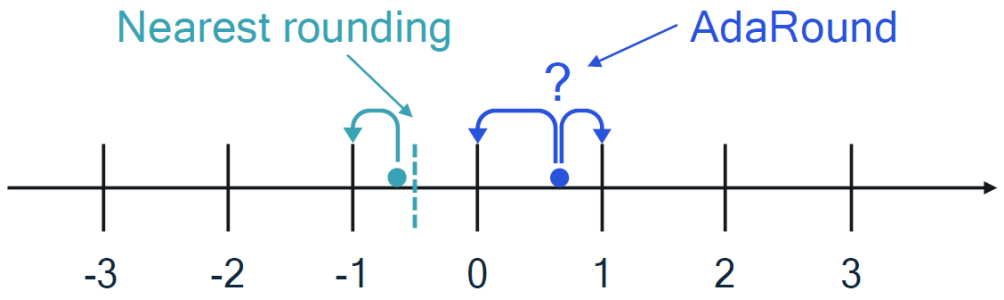
- Quantization Granularity
- Dynamic Range Clipping
- Rescaling
- **Rounding**



Adaptive Rounding for PTQ

AdaRound

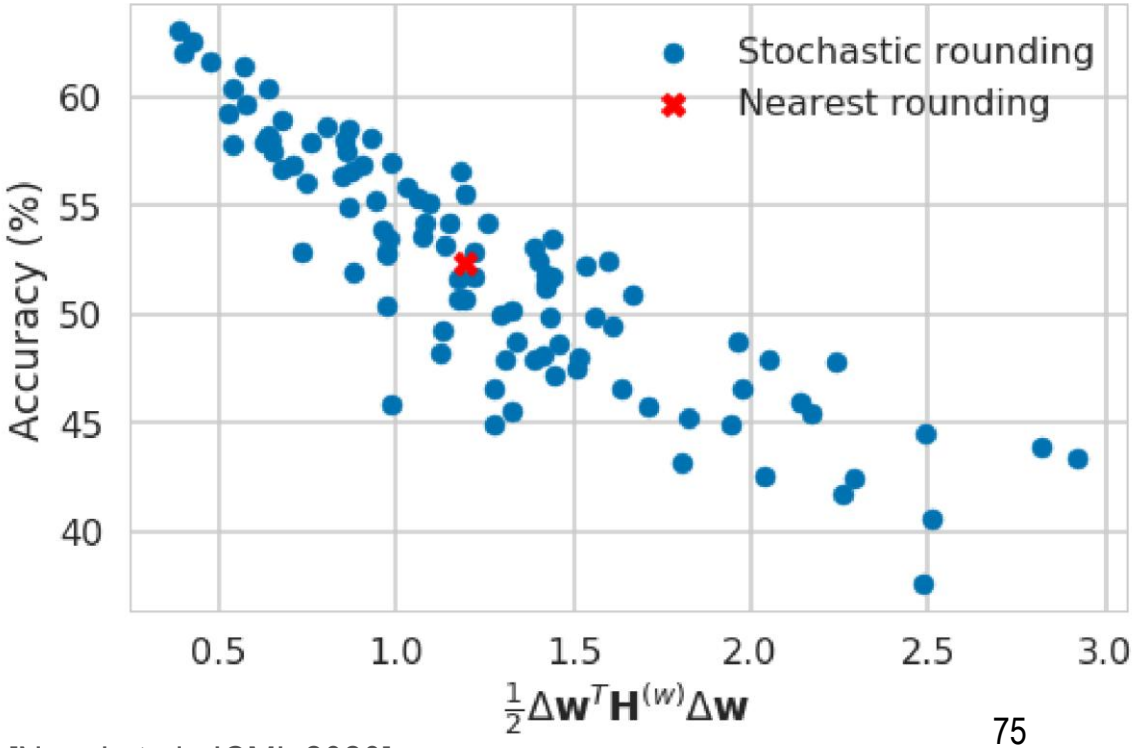
- **Rounding-to-nearest is sub-optimal**



What kind of rounding is optimal?

- Rounding that best reconstructs the original activation

- Weights are correlated with each other.
- The best rounding for each weight is not the best rounding for the whole tensor.
- AdaRound can only be used on quantizing weights.



Up or Down? Adaptive Rounding [Nagel et al., ICML 2020]

Adaptive Rounding for PTQ

Instead of $\lfloor w \rceil$, we want to choose between $\{\lfloor w \rfloor, \lceil w \rceil\}$ for best reconstruction:

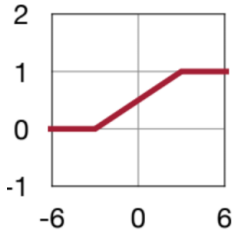
$$\widetilde{w} = \lfloor \lfloor w \rfloor + \delta \rceil, \delta \in [0, 1]$$

AdaRound applies a learning-based method to optimize the following equation:

$$\boldsymbol{argmin_V} \|\boldsymbol{Wx} - \widetilde{\boldsymbol{W}}\boldsymbol{x}\|_{\boldsymbol{F}}^{\boldsymbol{2}} + \boldsymbol{\lambda} f_{reg}(\boldsymbol{V})$$

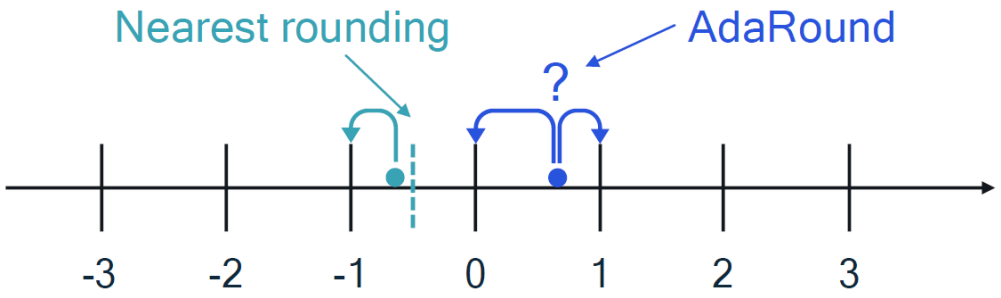
$$\longrightarrow \quad \boldsymbol{argmin_V} \|\boldsymbol{Wx} - \lfloor \lfloor w \rfloor + \boldsymbol{h}(\boldsymbol{V}) \rceil \boldsymbol{x}\|_{\boldsymbol{F}}^{\boldsymbol{2}} + \boldsymbol{\lambda} f_{reg}(\boldsymbol{V})$$

- $\boldsymbol{x}$: Input of the layer, $\boldsymbol{V}$: a random variable with the same shape
- $\boldsymbol{h}()$: a function to map distribution to (0, 1). (rectified sigmoid)
- $f_{reg}(\boldsymbol{V})$: a regularization term that encourages $h(\boldsymbol{V})$ to be binary



Up or Down? Adaptive Rounding [Nagel et al., ICML 2020]

AdaRound Accuracy Comparison



Optimization	#bits W/A	ResNet18	MobileNetV2
FP16	32/32	69.68	71.72
DFQ	8/8	69.7	71.2
Nearest	4/32	23.99	8.09
DFQ	4/8	38.98	46.57
AdaRound	4/8	**68.55**	**69.25**

Results for models in terms of ImageNet validation accuracy (%)

Up or Down? Adaptive Rounding [Nagel et al., ICML 2020]

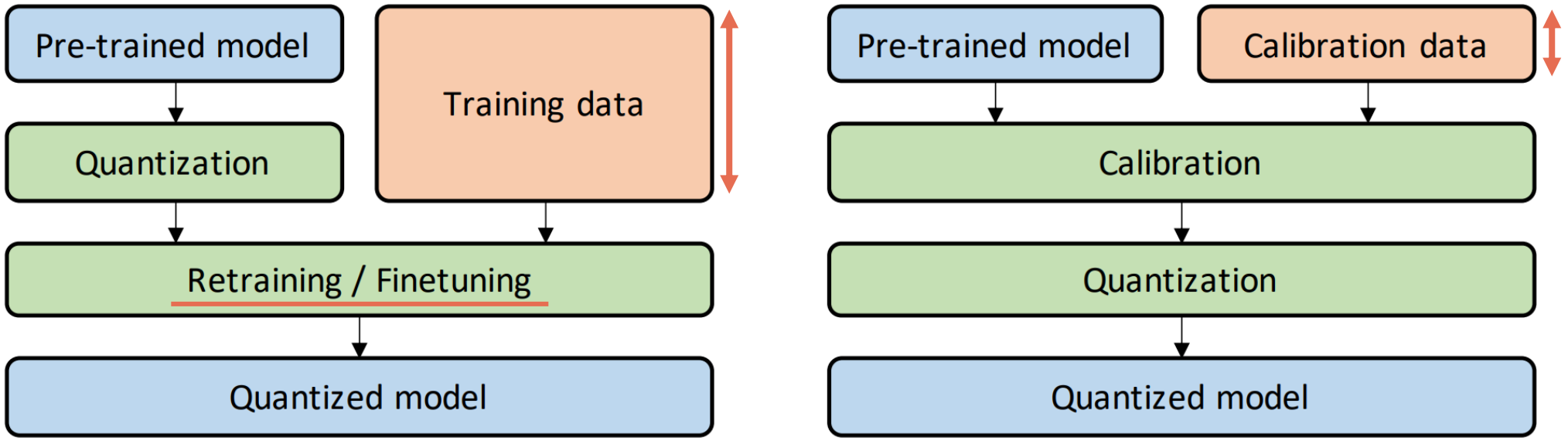
Quantization-Aware Training (QAT)

Can we train the model while taking quantization into consideration?



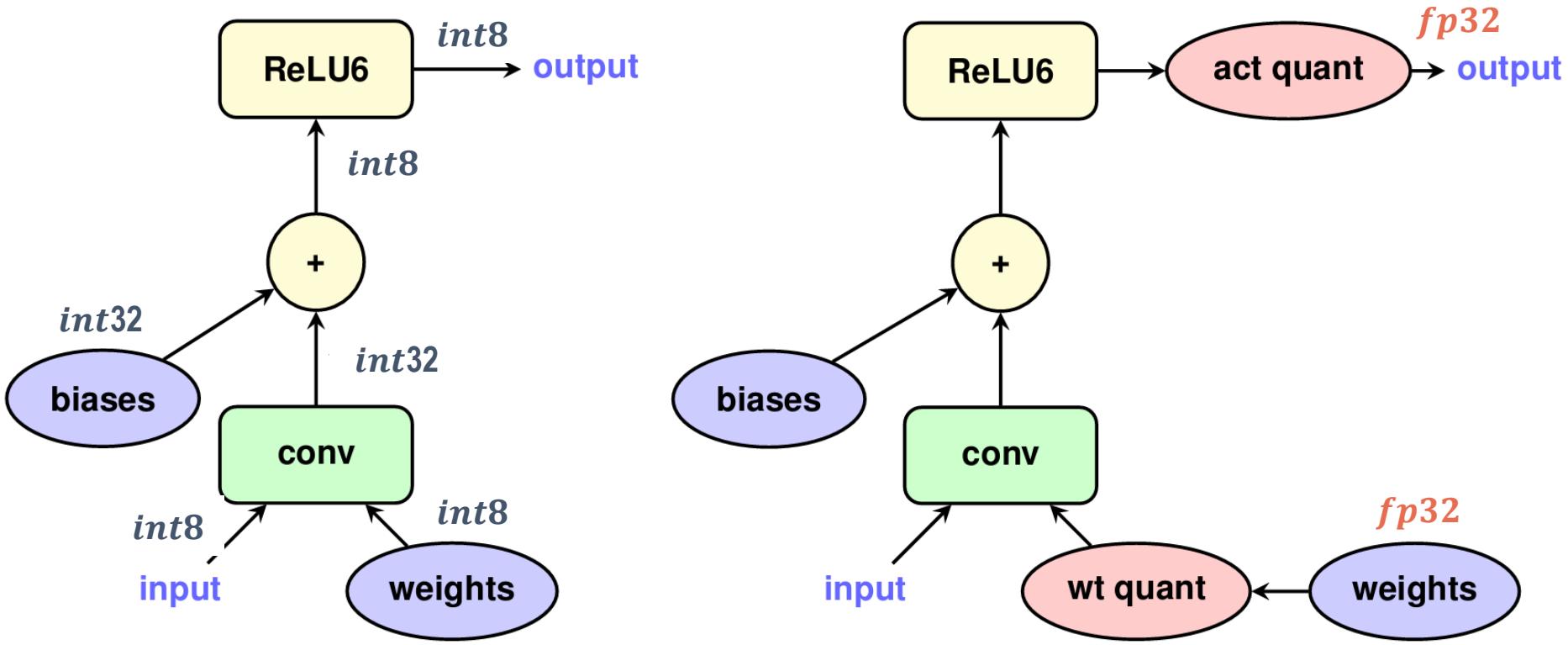
Quantization-Aware Training (QAT)

QAT vs. PTQ



Quantization-Aware Training (QAT)

- Train model while taking quantization into consideration



(a) Integer-arithmetic-only inference

(b) Training with simulated quantization

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Straight-Through Estimator (STE)

- **Train model while taking quantization into consideration**

$$\boldsymbol{Q(W)} = clip(\left\lfloor \frac{\boldsymbol{W}}{S} \right\rfloor, q_{min}, q_{max})$$

Quantization is discrete-valued, and thus the derivative is 0 almost everywhere.

$$\frac{\partial Q(W)}{\partial \mathrm{W}} = 0$$

- The network will learn nothing since gradients become 0 and weights won't update.

$$g_W = \frac{\partial L}{\partial \mathrm{W}} = \frac{\partial L}{\partial Q(W)} \cdot \frac{\partial Q(W)}{\partial \mathrm{W}} = 0$$

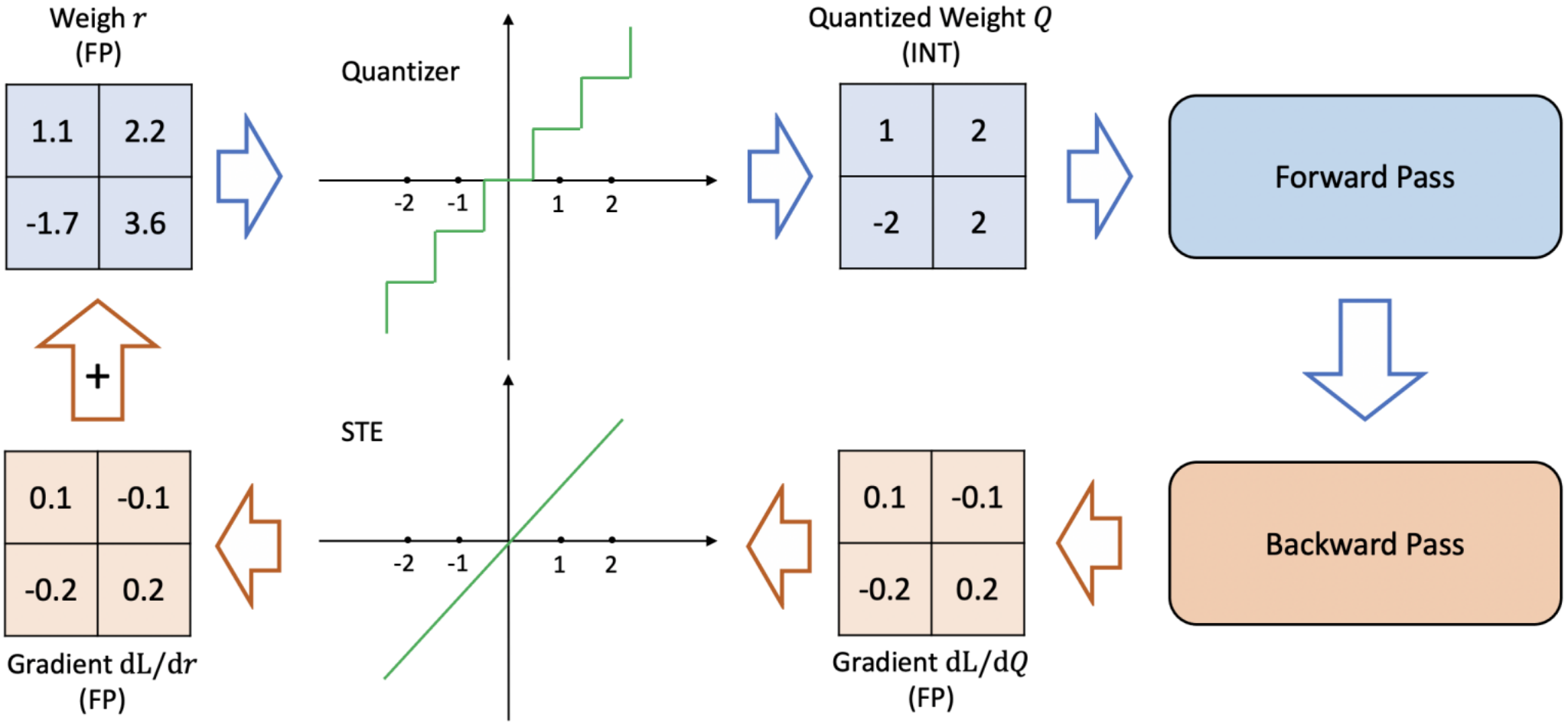
- **STE** passes the gradients through quantization as if it is an identity function.

$$g_W = \frac{\partial L}{\partial \mathrm{W}} = \frac{\partial L}{\partial Q(W)}$$



Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio et al., arXiv 2013]

Straight-Through Estimator (STE)



Survey of Quantization Methods for Efficient NN inference [Amir Gholami et al., Book: Low-Power Computer Vision]

Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio et al., arXiv 2013]

INT8 Linear Quantization-Aware Training

Experimental results (ImageNet)

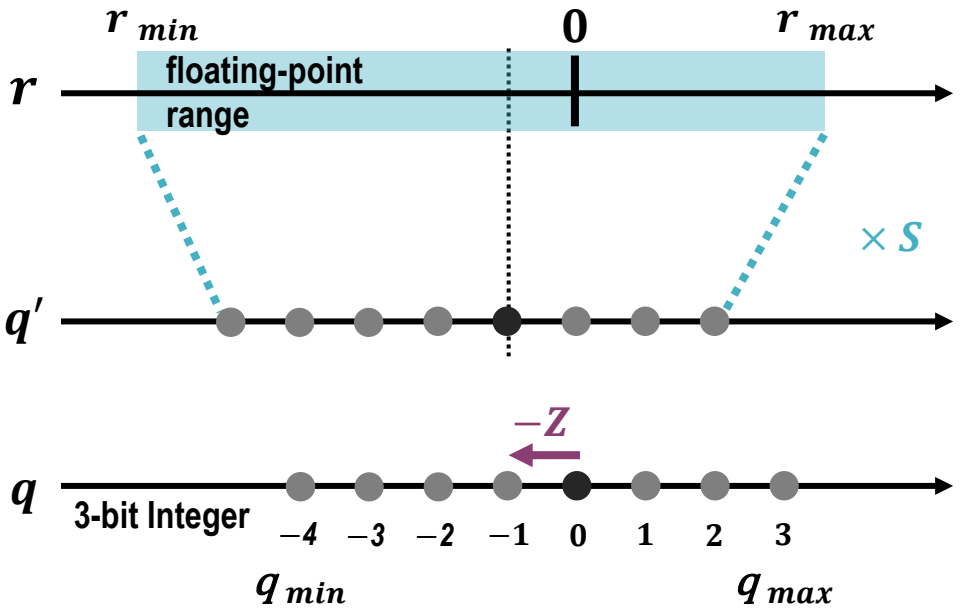
	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNET-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]

Summary of this Chapter

- The basic concept of neural network quantization, is to remap weights and activations into a limited discrete set of numbers

- Two common quantization methods
  - K-means-based Quantization
  - Linear Quantization



Summary of this Chapter

- Post-Training Quantization (PTQ) quantizes an already-trained floating-point model.
  - Per-tensor vs. per-channel vs. group-wise
  - How to determine the dynamic range for activation
- Quantization-Aware Training (QAT) emulates inference-time quantization during training/fine-tuning.
  - Straight Through Estimator (STE)

